

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

Hemapriya H. A. N. S

(IT22099518)

BSc (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

Hemapriya H. A. N. S

(IT22099518)

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of
Science (Hons) in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

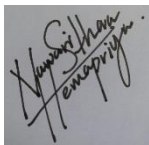
August 2025

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature



Date

25/04/2026

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor



Date

25/04/2026

ABSTRACT

This research presents the design and implementation of an AI-powered Visual Perception Component for the Human-Like Agent for Screen-Aware Task Automation system. The primary function of this component is to observe digital interfaces through screenshots or live camera feeds and transform raw visual input into structured representations of user interface (UI) elements suitable for downstream automated interaction.

The module employs a multimodal vision-language reasoning approach using Google Gemini, combined with image preprocessing and validation techniques, to detect and classify visible interface components such as buttons, text fields, menus, tabs, icons, labels, and navigation controls. The generated output includes semantic labels, confidence scores, spatial coordinates, and bounding regions in a machine-readable JSON format.

The system is designed to operate in real-time conjunction with higher-level planning and execution modules. By continuously analyzing visual scenes, the component provides updated contextual information required for intelligent task execution, allowing downstream agents to adapt dynamically to UI changes, incomplete states, hidden elements, or unexpected interface transitions.

Unlike conventional automation frameworks that depend on internal software APIs, DOM structures, accessibility metadata, or source-level integration, the proposed approach relies entirely on external visual observation. This enables effective operation in black-box environments where traditional automation methods are restricted, unavailable, or impractical. As a result, the component can be applied across desktop systems, browser environments, legacy applications, and third-party platforms without requiring system modification.

By integrating computer vision preprocessing, multimodal semantic reasoning, structured validation, and session-level artifact generation, the proposed Visual Perception Component demonstrates a flexible and extensible solution for intelligent GUI understanding. Experimental evaluation within controlled environments confirms its capability to generate accurate, structured, and automation-ready outputs across multiple interface scenarios.

Keywords – Visual Perception, Computer Vision, GUI Automation, Vision-Language Model, Screen Understanding, Black-Box Testing

ACKNOWLEDGEMENT

I would like to express my deep gratitude for all the people who helped me to continue my 4th year research project. First, I greatly appreciate our Research Project supervisor Mr. Vishan Jayasinghearachchi, who provided the guidance and encouragement to continue this project successfully. I am also grateful to our co-supervisor Ms. Kaushalya Rajapaksa for the valuable and constructive suggestions given during the planning of our research work.

I also acknowledge my Research Project leader Annesiyani Srikanthan and other group members who have been supportive in pursuing these goals. Finally, I would like to give my heartfelt gratitude to my parents and friends who helped me and provided unending inspiration.

TABLE OF CONTENTS

DECLARATION	i
ABSTRACT.....	ii
ACKNOWLEDGEMENT.....	iv
LIST OF FIGURES.....	vii
LIST OF TABLES.....	viii
LIST OF ABBREVIATIONS	ix
1. INTRODUCTION.....	1
1.1 General Introduction.....	1
1.2 Background Literature	2
1.3 Limitation in the Current Approach	4
1.4 Research Gap.....	5
2. RESEARCH PROBLEM	9
2.1 Problem Context.....	9
2.2 Problem Statement	10
2.3 Core Research Problem	11
3. RESEARCH OBJECTIVES.....	13
3.1 Main Objective.....	13
3.2 Specific Objective	14
3.2.1 To develop an image acquisition and preprocessing pipeline for screen understanding	14
3.2.2 To implement a multimodal screen understanding model for generalized UI detection	15
3.2.3 To extract structured machine-readable UI metadata from visual outputs	15
3.2.4 To design validation mechanisms for reliability and safe automation	15
3.2.5 To support debugging, monitoring, and explainability through artifact generation	15
3.2.6 To evaluate performance in terms of latency, coverage, and operational usability	16
3.2.7 To create a modular API-based perception service for integration with autonomous agents	16
3.2.8 To contribute a generalized perception framework for future screen-aware intelligent systems	16
4. METHODOLOGY	17
4.1 System Architecture	17
4.2 Specific Component Description	22
4.2.1 Purpose.....	22
4.2.2 Workflow Overview.....	23
4.2.3 Input Acquisition Stage	24
4.2.4 Image Preprocessing Stage.....	25
4.2.5 Semantic UI Detection Stage	25
4.2.6 Structured Output Generation	26
4.2.7 Validation and Reliability Stage.....	27
4.2.8 Debugging and Session Management	27

4.2.9	Previous Prototype Approaches	28
4.2.10	Final Operational Outcome	28
4.3	Design System Diagrams.....	29
4.3.1	DFD Diagram	29
5.	IMPLEMENTATION AND TESTING.....	31
5.1	Implementation.....	31
5.1.1	Vision Service Overview.....	31
5.1.2	Gemini Backend Integration	32
5.1.3	Image Input and Preprocessing Pipeline	35
5.1.4	UI Detection and Structured Output Generation	38
5.1.5	Validation, Coordinates, and Session Management	40
5.1.6	Model Selection and Operational Decision	43
5.2	Testing	44
5.2.1	Test Plan and Strategy	44
5.2.2	Test Case Design	45
6.	RESULTS AND DISCUSSIONS	54
6.1	Results Analysis	54
6.1.1	Overall Execution Outcomes.....	54
6.1.2	Quantitative Performance Summary	56
6.1.3	Accuracy and Output Structure Quality	57
6.1.4	Robustness Under Edge Condition.....	105
6.1.5	Comparative Behavior Across Scenario Types	105
6.1.6	Practical Result Interpretation	106
6.2	Research Finding.....	107
6.2.1	Visual-language reasoning enables practical screen understanding	107
6.2.2	Structured output improves downstream integration.....	107
6.2.3	Validation layers increase reliability	107
6.2.4	Semantic generalization outperformed fixed detectors	107
6.2.5	Speed-quality trade-offs exist.....	107
6.2.6	Black-box automation is feasible	107
6.2.7	Modular architecture supports extensibility	108
6.3	Discussion	108
6.3.1	Interpretation Against Research Objectives	108
6.3.2	Effectiveness of the Proposed Approach.....	108
6.3.3	Implication of Human-Computer Interaction	108
6.3.4	Reliability-Latency Trade-off.....	109
6.3.5	Workflow Continuity as a Limitation and Opportunity	109
6.3.6	Trust, Safety and Explainability	109
6.3.7	Threats to Validity	110
6.3.8	Practical Deployment Implications	110
7.	CONCLUSIONS	110
8.	REFERENCES.....	113
9.	APPENDICES	115

LIST OF FIGURES

Figure 4.1: Overall system architecture diagram	18
Figure 4.2: High-level layered architecture diagram	21
Figure 4.3: Vision processing workflow	24
Figure 4.4: Visual perception data flow diagram	30
Figure 5.1: Vision endpoint flow diagram	32
Figure 5.2: Gemini client implementation flow	33
Figure 5.3: Gemini client implementation	34
Figure 5.4: Gemini API configuration	34
Figure 5.5: Prompt used for UI detection.....	35
Figure 5.6: Image Preprocessing Pipeline.....	37
Figure 5.7: OpenCV processing	38
Figure 5.8: Sample request and response	39
Figure 5.9: Sample response payload.....	40
Figure 5.10: Output validation pipeline	41
Figure 5.11: Snapshot of the session folder	42
Figure 5.12: Debug overlay image.....	43
Figure 6.1: Detected UI Elements with Bounding overlays.....	55
Figure 6.2: Vision API Request Payload through Postman	56

LIST OF TABLES

Table 4.1: Technologies and Techniques Used in the Proposed System	20
Table 5.1: Summarizes comparative observation	44
Table 5.2: Verify login screen UI detection	45
Table 5.3: Verify missing UI element handling	46
Table 5.4: Verify complex dashboard UI detection	46
Table 5.5: Verify JSON output correctness	50
Table 5.6: Verify session artifact generation	51
Table 6.1: Vision Component Performance Summary	57

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
DOM	Document Object Model
EasyOCR	Easy Optical Character Recognition
ESP32	Espressif Systems 32-bit Microcontroller
FastAPI	Fast Application Programming Interface
GUI	Graphical User Interface
HID	Human Interface Device
JSON	JavaScript Object Notation
OCR	Optical Character Recognition
OpenCV	Open-Source Computer Vision Library
PaddleOCR	Paddle Optical Character Recognition
REST	Representational State Transfer
RPA	Robotic Process Automation
UI	User Interface
VLM	Vision-Language Model
XML	Extensible Markup Language
YOLOv8	You Only Look Once version 8

1. INTRODUCTION

1.1 General Introduction

The rapid advancement of intelligent automation technologies has significantly transformed the way repetitive digital tasks are executed across modern software environments. Conventional automation solutions commonly rely on internal application programming interfaces (APIs), Document Object Model (DOM) structures, accessibility trees, or predefined software hooks to interact with user interfaces. Although such approaches perform effectively in controlled environments, they become limited when applied to legacy systems, secure platforms, virtualized environments, or third-party applications where internal system access is unavailable or intentionally restricted.

To address these limitations, the **Screen-Aware Task Agent** was developed as a human-like automation framework capable of interacting with graphical user interfaces through external observation and hardware-level execution. Rather than relying on internal software access, the system observes the screen visually, interprets interface states, and performs actions in a manner like a human user. This black-box interaction model enables automation across heterogeneous platforms while preserving realism and platform independence.

Within this architecture, the **Visual Perception Component** functions as the core sensory subsystem of the overall agent. It is responsible for observing the target screen, understanding visible interface content, identifying interactable elements, and transforming raw visual data into structured machine-readable representations. The output of this module enables downstream planning and execution components to make informed automation decisions based on the actual state of the screen.

The implemented perception system follows a multi-stage pipeline consisting of frame acquisition, image preprocessing, multimodal screen understanding, output validation, and structured result generation. Initially, screen frames are captured through external camera feeds or direct input sources. These frames are then preprocessed using image

enhancement techniques such as resizing, normalization, cropping, and noise reduction. The processed image is subsequently analyzed using the production multimodal perception backend based on **Gemini**, which performs semantic understanding of the user interface and identifies visible UI components such as buttons, input fields, labels, menus, icons, tabs, and browser controls.

Unlike traditional object detection systems that depend on fixed training classes, the Gemini-based architecture enables generalized perception across previously unseen interfaces. This allows the system to understand new layouts, varying themes, dynamic content, and diverse software applications without retraining task-specific detection models. Structured outputs are returned in JSON format containing labels, UI types, confidence values, coordinates, and optional bounding boxes, which can be consumed directly by planning and automation modules.

Earlier development phases of the project explored locally hosted multimodal models through **Ollama**, OCR pipelines, and **YOLOv8**-based detection strategies. While these approaches demonstrated lower latency and acceptable performance under constrained conditions, they exhibited limitations in broad semantic interpretation and cross-application generalization. Consequently, the final production architecture standardized on Gemini-based visual perception due to stronger reasoning capability, improved flexibility, and more reliable structured outputs.

The Visual Perception Component therefore represents a critical contribution to the Screen-Aware Task Agent by enabling intelligent screen understanding in real-world black-box environments. Through the integration of computer vision, multimodal reasoning, and structured validation, the module establishes a practical foundation for adaptive and scalable UI automation.

1.2 Background Literature

Graphical User Interface (GUI) automation has been widely studied in the domains of software engineering, robotic process automation, and intelligent user assistance systems. Early automation approaches such as scripted macro tools, Selenium-based web automation, and rule-driven Robotic Process Automation (RPA) frameworks operate

effectively when internal application structures are accessible. These systems typically depend on stable identifiers, DOM trees, accessibility metadata, or APIs for interaction accuracy. However, such assumptions are not held in many real-world environments where systems are closed, legacy-based, remote, or visually rendered without structured hooks.

To overcome these restrictions, researchers introduced vision-based automation methods that rely on camera feeds instead of internal metadata. Computer vision techniques such as template matching, contour analysis, Optical Character Recognition (OCR), and supervised object detection have been applied to identify interface components directly from pixels. OCR systems such as **Tesseract OCR** enabled extraction of textual content from software screens, improving automation in form filling and menu navigation tasks.

Subsequent advances in deep learning led to the use of Convolutional Neural Networks (CNNs) and object detection architectures such as **YOLO** for UI element localization. These systems could identify buttons, icons, text fields, and menus at high speed, making them suitable for low-latency applications. However, their performance depends heavily on labeled training datasets and predefined classes. As a result, they often struggle when exposed to unseen interface designs, custom enterprise systems, or rapidly changing UI layouts.

More recently, Vision-Language Models (VLMs) and multimodal large language models have significantly expanded the capabilities of screen understanding systems. Models such as **Gemini** can process both visual and textual inputs while performing semantic reasoning over interface content. Rather than detecting only predefined objects, such systems can interpret contextual meaning, infer likely interactable elements, understand screen purpose, and generate structured representations suitable for downstream reasoning.

This evolution from fixed computer vision models toward generalized multimodal perception aligns closely with the requirements of adaptive UI automation. In dynamic black-box environments, perception systems must not only detect shapes and text but also

understand user interface semantics under uncertainty. Therefore, multimodal reasoning models have become increasingly relevant for next-generation automation systems.

1.3 Limitation in the Current Approach

Although the proposed Visual Perception Component demonstrates effective screen understanding capabilities, several practical limitations remain in the current implementation.

First, the system depends on visual quality of incoming frames. Low lighting conditions, motion blur, camera misalignment, glare, low-resolution images, and compression artifacts can negatively affect detection accuracy and semantic interpretation. Since the architecture supports external visual observation, environmental variability remains an operational challenge.

Second, the production Gemini-based pipeline introduces higher inference latency compared with lightweight local detectors. While generalized understanding is improved, response times typically range higher than specialized fast-path detectors. This may affect highly time-sensitive automation scenarios.

Third, multimodal reasoning systems may occasionally produce inconsistent bounding boxes, approximate coordinates, or confidence variations depending on screen complexity and prompt context. Therefore, output validation layers are necessary before execution.

Fourth, internet connectivity and cloud inference availability may influence runtime reliability when cloud-hosted production models are used. This differs from fully offline local detection architectures.

Fifth, although the system supports broad UI generalization, domain-specific enterprise applications with highly customized visual designs may still require prompt refinement or hybrid detection enhancements for optimal performance.

1.4 Research Gap

The field of intelligent GUI automation has experienced rapid growth through the emergence of robotic process automation, computer vision, and multimodal large language models. However, despite these advancements, a considerable research gap remains in the design of robust visual perception systems capable of supporting real-world screen-aware automation in heterogeneous black-box environments.

Traditional GUI automation frameworks such as Selenium, Appium, and UIAutomator rely heavily on internal application metadata including DOM trees, accessibility identifiers, XML hierarchies, or platform-level APIs. These methods are highly effective in controlling web or mobile testing scenarios where internal structures are exposed. Nevertheless, their applicability becomes significantly limited when deployed in environments such as legacy desktop systems, remote desktop sessions, virtual machines, third-party enterprise software, secure platforms, or applications that intentionally restrict programmatic access. In such situations, automation systems require purely visual interaction capabilities rather than internal software hooks [1], [2], [3].

To overcome this limitation, image-based automation tools such as SikuliX introduced pixel-level template matching approaches that allow systems to interact with applications based on screens. While these tools represent an important milestone in black-box automation, they are sensitive to scaling changes, theme variations, language differences, animation states, and minor UI redesigns. Consequently, template-driven systems often require frequent manual maintenance and demonstrate poor robustness under dynamic interface conditions [4].

The use of Optical Character Recognition (OCR) further improved visual automation by enabling extraction of textual content from screens. Technologies such as Tesseract OCR, EasyOCR, and PaddleOCR support recognition of labels, buttons, menus, and textual controls visible in software interfaces [5], [6], [7], [8]. However, OCR systems primarily focus on text recognition rather than comprehensive interface understanding. They may detect words successfully yet fail to infer whether a region represents an actionable button, disabled control, dropdown menu, tab navigation element, or composite widget.

Therefore, OCR alone is insufficient for complete automation reasoning in modern graphical environments.

Deep learning-based object detection introduced a new generation of GUI perception methods. Studies in GUI object detection demonstrated that neural models could localize buttons, text fields, icons, and structural components with improved speed and accuracy compared with classical image processing methods [9]. Architectures such as YOLO became especially attractive due to low inference latency and suitability for real-time automation systems. However, object detection methods depend strongly on labeled training datasets and predefined classes. As a result, their performance often degrades when applied to previously unseen enterprise applications, customized dashboards, uncommon design systems, or rapidly evolving user interfaces. In practical deployment, maintaining training datasets for every possible interface category becomes operationally expensive and difficult to scale [9].

Recent progress in Vision-Language Models (VLMs) and multimodal intelligent agents has shifted research toward generalized screen understanding. Systems such as CogAgent, ScreenAgent, CoCo-Agent, GUI-R1, TinyClick, and Multimodal Chain-of-Action agents demonstrate that models can interpret screen data, understand user goals, reason about graphical layouts, and propose executable interface actions through natural language instructions [10], [11], [12], [13], [14], [15]. These studies provide strong evidence that multimodal reasoning can surpass traditional fixed detectors by enabling semantic understanding of unseen interfaces.

Despite this progress, several unresolved research challenges remain.

First, many current VLM-based GUI agent studies are evaluated in controlled benchmark environments, browser sandboxes, mobile simulators, or curated datasets. Real production environments introduce substantially more difficult conditions including noisy camera input, reflections, varying resolutions, inconsistent fonts, multilingual interfaces, partial occlusion, remote desktop compression artifacts, and changing window layouts. Existing benchmark-focused studies do not fully address these operational constraints [11], [13], [14].

Second, many multimodal GUI agent systems focus primarily on planning and action reasoning, while comparatively less attention is given to the visual perception pipeline itself. In practical automation systems, perception must produce structured outputs such as UI type labels, confidence values, pixel coordinates, bounding boxes, visibility states, and machine-readable schemas that downstream execution modules can trust. Many research prototypes describe high-level capabilities but provide limited detail regarding production-grade perception validation and coordinate reliability [10], [12], [15].

Third, there is limited comparative research on transitioning from local lightweight perception models to cloud-based generalized multimodal systems. Lightweight local detectors may offer low latency and privacy advantages, while cloud multimodal systems may offer stronger semantic understanding and better zero-shot generalization. However, systematic studies analyzing this trade-off in real automation pipelines remain insufficient. This is particularly relevant for engineering decisions regarding cost, response time, maintainability, scalability, and deployment readiness [9], [16], [14].

Fourth, current literature provides limited discussion on hybrid perception architectures that combine multiple capabilities such as image preprocessing, OCR enhancement, multimodal reasoning, coordinate verification, schema validation, confidence scoring, debug artifact generation, and API-based orchestration into a unified microservice deployment model. In real enterprise environments, such integration is essential for maintainability and operational success.

Fifth, explainability and trust remain underexplored in screen automation perception systems. For practical deployment, automation systems must not only detect elements correctly but also provide transparent reasoning signals such as confidence scores, missing element reports, and traceable outputs. Many existing systems emphasize benchmark success rates without sufficient focus on auditable production behavior.

Sixth, long-term adaptability remains a major gap. Modern software interfaces continuously evolve through redesigns, dark mode themes, responsive layouts, localization changes, and feature experimentation. Systems based purely on fixed labels

or static templates require frequent retraining or manual updates. More generalized perception systems are therefore needed to support sustainable automation over time.

Therefore, a clear research gap exists in the development of a production-ready visual perception framework that can:

- Understand previously unseen interfaces without retraining
- Detect actionable UI components using semantic reasoning
- Return structured coordinates and validated machine-readable outputs
- Operate under noisy real-world visual conditions
- Balance latency, scalability, and perception accuracy
- Support downstream autonomous execution modules
- Provide explainable and auditable outputs for safe automation
- Function reliably in black-box cross-platform environments

The proposed Visual Perception Component addresses this gap through a Gemini-centered multimodal perception architecture supported by image preprocessing, structured schema validation, coordinate verification, confidence estimation, debug overlay generation, and service-based deployment. Unlike traditional fixed detectors, the proposed system emphasizes generalized UI understanding while maintaining practical integration with downstream task automation modules. This contributes toward the next generation of adaptive, scalable, and real-world deployable screen-aware intelligent automation systems.

2. RESEARCH PROBLEM

2.1 Problem Context

The increasing dependence on digital systems across education, healthcare, finance, administration, manufacturing, and enterprise operations has created a strong demand for intelligent automation solutions capable of performing repetitive computer-based tasks efficiently. Organizations increasingly rely on software applications for document handling, customer support, data entry, workflow approvals, scheduling, communication, and system monitoring. Many of these tasks continue to require human operators to manually interact with graphical user interfaces such as desktop applications, web portals, virtual machines, and remote desktop environments. As the volume of such tasks grows, manual execution becomes time-consuming, error-prone, and economically inefficient.

Conventional automation methods have attempted to solve this problem through rule-based scripting and Robotic Process Automation (RPA) platforms. These systems typically interact with applications using internal APIs, browser Document Object Model (DOM) structures, accessibility metadata, test automation hooks, or predefined selectors. While highly effective in structured and controlled environments, these approaches become unreliable or unusable when deployed in real-world heterogeneous software ecosystems. Many organizations still operate legacy desktop applications, custom enterprise systems, Citrix-based environments, secure remote sessions, or third-party tools where internal automation interfaces are unavailable or intentionally restricted. In such cases, traditional automation cannot easily identify interface elements or perform stable interaction.

At the same time, modern software interfaces are becoming increasingly dynamic and visually complex. Applications frequently change layouts, support dark mode themes, responsive designs, multilingual content, animated controls, nested menus, and adaptive workflows. Static rule-based automation methods often fail when minor UI changes occur. Maintaining selectors, scripts, and brittle automation rules becomes expensive over time. Therefore, there is growing interest in screen-aware intelligent agents that

interact with systems as a human would by visually observing the screen, understanding interface content, and executing actions externally.

For such an agent to function effectively, robust visual perception becomes essential. The automation system must recognize visible buttons, text fields, labels, icons, menus, tabs, and system controls directly from screen pixels. It must also understand context, infer which elements are actionable, determine screen state, and return accurate coordinates for execution modules. This requirement transforms the automation challenge from a simple scripting problem into a computer vision and multimodal reasoning problem.

Although recent advances in OCR, deep learning object detection, and Vision-Language Models (VLMs) have created new opportunities for intelligent screen understanding, many existing solutions remain limited. OCR focuses mainly on text extraction, object detectors depend on predefined labels, and some multimodal systems remain experimental or benchmark oriented. There remains a practical need for a production-ready visual perception module capable of generalized UI understanding across diverse real-world environments. This context forms the basis of the present research.

2.2 Problem Statement

Current GUI automation solutions face significant limitations when applied to real-world black-box software environments where internal application metadata is inaccessible. Many traditional automation systems depend on selectors, APIs, accessibility trees, XML hierarchies, or browser DOM structures to identify and interact with user interface components. When such internal structures are unavailable, unstable, or intentionally blocked, these methods fail to provide reliable automation capability.

Even in environments where partial automation access exists, frequent software updates, UI redesigns, theme changes, language localization, dynamic content loading, and responsive layouts often break predefined scripts. As a result, organizations must repeatedly maintain and rewrite automation logic, increasing cost and reducing long-term sustainability. This issue is particularly severe in enterprise environments containing multiple software platforms developed using different technologies.

Alternative image-based approaches such as template matching tools offer some black-box capability, yet they are highly sensitive to visual variations such as scaling, color changes, font differences, animation states, and partial occlusions. Likewise, OCR systems can detect visible text but do not always understand structural meaning or whether recognized text belongs to a clickable button, passive label, tab, input box, or disabled control.

Deep learning object detectors such as YOLO-based systems improve speed and localization accuracy, but their dependency on labeled datasets and predefined classes limits adaptability. New software layouts, uncommon interface widgets, custom enterprise dashboards, and previously unseen UI designs may require retraining or manual annotation. This creates scalability challenges when automation must operate across many unknown systems.

Recent multimodal models have shown strong reasoning ability over screen data and interfaces. However, many existing research systems prioritize benchmark performance or high-level action planning rather than practical deployment concerns. Real automation systems require structured machine-readable outputs, accurate pixel coordinates, confidence scores, validation pipelines, API integration, and stable operation under noisy production conditions.

Therefore, the key problem is that existing solutions do not sufficiently combine generalized screen understanding, structured UI detection, coordinate precision, operational robustness, and deployable architecture into a single practical perception framework. Without solving this problem, fully autonomous screen-aware task agents cannot perform reliably across diverse real-world software environments.

2.3 Core Research Problem

Based on the identified limitations, the core research problem of this study can be stated as follows:

How can a production-ready visual perception component be designed and implemented to enable a screen-aware intelligent agent to accurately understand

diverse graphical user interfaces, detect actionable UI elements, and generate structured execution-ready outputs in black-box real-world environments where traditional automation hooks are unavailable?

This central problem contains several interconnected technical challenges.

First, the system must achieve generalized perception across previously unseen interfaces without requiring retraining for each new application. Unlike traditional object detection systems that recognize fixed categories only, the proposed solution must understand a broad variety of layouts, widgets, themes, and screen states.

Second, the system must convert raw visual observations into structured data representations that downstream automation modules can consume directly. This includes UI element type, descriptive labels, confidence values, bounding boxes, pixel coordinates, and state information.

Third, the perception pipeline must remain robust under real operational conditions such as noisy screen data, different screen resolutions, compressed remote desktop feeds, varying fonts, low contrast interfaces, and changing visual themes.

Fourth, the solution must balance intelligence with practical deployment constraints. While advanced multimodal reasoning models may improve generalization, the architecture must still support acceptable response times, maintainability, scalability, and service integration.

Fifth, the system must support transparency and reliability through validation mechanisms such as schema checking, coordinate verification, confidence filtering, and debug artifact generation. These safeguards are critical because inaccurate perception outputs can directly lead to failed or unsafe automation actions.

To address these challenges, this research proposes a Gemini-centered Visual Perception Component that integrates image preprocessing, multimodal screen understanding, structured JSON generation, coordinate validation, and modular API deployment. The

intention is to bridge the gap between academic GUI understanding research and practical real-world intelligent automation systems.

3. RESEARCH OBJECTIVES

3.1 Main Objective

The main objective of this research is to design, develop, and evaluate a production-ready **Visual Perception Component** for a Screen-Aware Task Agent that can intelligently observe graphical user interfaces, understand visible screen content, identify actionable user interface elements, and generate structured outputs for autonomous task execution in black-box computing environments.

Modern automation systems often depend on internal software hooks such as APIs, DOM trees, accessibility metadata, or predefined selectors. However, many real-world environments do not expose such internal structures. Therefore, the primary aim of this study is to create a perception system that enables automation through external visual observation in a human-like manner. Instead of relying on software internals, the proposed module interprets the screen directly from images or captured frames and provides machine-readable information that downstream planning and control modules can use to interact with applications.

Another major objective is to achieve generalized interface understanding across a wide range of software platforms including desktop applications, web systems, remote desktops, legacy tools, and custom enterprise software. The solution is expected to function effectively even when interface layouts, themes, colors, fonts, and control placements differ significantly. This objective is important because traditional detector-based systems often struggle when exposed to previously unseen interfaces.

The research also aims to integrate modern multimodal reasoning capabilities into practical automation pipelines. By utilizing a Gemini-centered visual understanding architecture, the component seeks to move beyond simple OCR or fixed object detection

toward semantic interpretation of screen elements. This includes identifying whether visible regions represent buttons, text fields, tabs, menus, labels, icons, navigation controls, or other interactable objects.

Furthermore, the main objective includes transforming perception outputs into structured and reliable formats suitable for automation execution. The module is intended to return validated JSON responses containing UI element types, descriptive labels, confidence scores, coordinates, bounding boxes, and optional state information. This bridges the gap between visual understanding and executable automation logic.

Finally, this research seeks to demonstrate that a modular visual perception microservice can serve as a scalable foundation for future intelligent GUI agents. Through experimentation, performance evaluation, and architectural validation, the study aims to contribute a practical framework for next-generation screen-aware automation systems.

3.2 Specific Objective

To accomplish the main objective, the research is divided into several specific objectives that collectively address the technical, operational, and evaluative requirements of the Visual Perception Component.

3.2.1 To develop an image acquisition and preprocessing pipeline for screen understanding

One specific objective is to build a robust front-end pipeline capable of receiving captured frames, or screen images and preparing them for perception analysis. This includes frame loading, resizing, cropping, normalization, denoising, contrast enhancement, and image formatting. Effective preprocessing is essential because poor image quality can negatively affect downstream recognition accuracy. The system must therefore standardize visual input under varying screen resolutions and quality conditions.

3.2.2 To implement a multimodal screen understanding model for generalized UI detection

Another important objective is to integrate a modern multimodal model capable of analyzing graphical user interfaces semantically rather than relying only on fixed classes. Gemini-centered architecture is intended to interpret screen data and identify a wide range of visible elements such as buttons, forms, menus, links, tabs, icons, side panels, labels, and browser controls. This objective focuses on achieving adaptability across diverse unseen interfaces without requiring retraining for each software platform.

3.2.3 To extract structured machine-readable UI metadata from visual outputs

The research further aims to transform model responses into structured automation-ready data. This includes extracting UI element categories, descriptive names, confidence values, bounding boxes, pixel coordinates, and state indicators such as enabled, disabled, selected, or visible conditions when available. Structured outputs are necessary because downstream task planning and execution modules require deterministic data rather than free-form textual descriptions.

3.2.4 To design validation mechanisms for reliability and safe automation

Since inaccurate perception can lead to failed clicks, wrong selections, or unsafe actions, another objective is to implement validation mechanisms within the perception pipeline. These include schema validation, coordinate verification, confidence filtering, duplicate removal, and consistency checks. Such safeguards improve trustworthiness and operational stability of the automation agent.

3.2.5 To support debugging, monitoring, and explainability through artifact generation

A practical production system requires visibility into its internal behavior. Therefore, one specific objective is to generate session logs, overlays, marked screen data, and structured debug artifacts that allow developers to inspect detected UI regions and diagnose failures.

This objective improves maintainability and supports continuous enhancement of the perception component.

3.2.6 To evaluate performance in terms of latency, coverage, and operational usability

The study also seeks to assess the implemented component using relevant performance indicators. These include average response time, UI element coverage, output consistency, coordinate usefulness, and suitability for real automation workflows. Comparative insights from earlier YOLO-based and local VLM experiments are also relevant to demonstrate the reasoning behind the final production architecture.

3.2.7 To create a modular API-based perception service for integration with autonomous agents

Another specific objective is to deploy the perception module as an independent service using FastAPI-based architecture. This allows seamless communication with planning agents, execution controllers, and external orchestration systems. A modular service-oriented design improves scalability, maintainability, and future extensibility.

3.2.8 To contribute a generalized perception framework for future screen-aware intelligent systems

Finally, the research aims to contribute not merely to a single software module, but a reusable framework for future intelligent agents that must interact with digital systems visually. By demonstrating the feasibility of multimodal perception in real automation scenarios, the study contributes toward broader research in human-like computing agents, digital robotics, and adaptive GUI intelligence.

4. METHODOLOGY

This chapter presents the methodology adopted for designing and implementing the proposed Screen-Aware Task Agent, with particular emphasis on the Visual Perception Component. The methodology explains the overall system architecture, internal component interactions, perception workflow, supporting technologies, and design models used to achieve intelligent automation in black-box environments. Since the proposed solution integrates software intelligence, computer vision, multimodal reasoning, and hardware actuation, a structured architectural methodology was essential to ensure modularity, scalability, maintainability, and real-world usability.

The system was developed using a layered microservice approach in which perception, planning, orchestration, and execution modules operate as loosely coupled services. This design enables independent enhancement of individual modules while preserving overall system functionality. The following sections describe the architecture and implementation strategy in detail.

4.1 System Architecture

The proposed Screen-Aware Task Agent was architected as an intelligent closed-loop automation system capable of receiving user commands, understanding screen states, generating actions, and physical interacting with a target computer. The architecture was designed to mimic the workflow of a human operator who observes a screen, reasons about what to do next, and performs mouse or keyboard actions accordingly. The system consists of multiple interconnected layers including the user interface layer, orchestration layer, perception layer, reasoning layer, and hardware actuation layer. Each layer performs a specific responsibility while communicating through structured APIs and control protocols. This modular architecture ensures separation of concerns and enables future scalability.

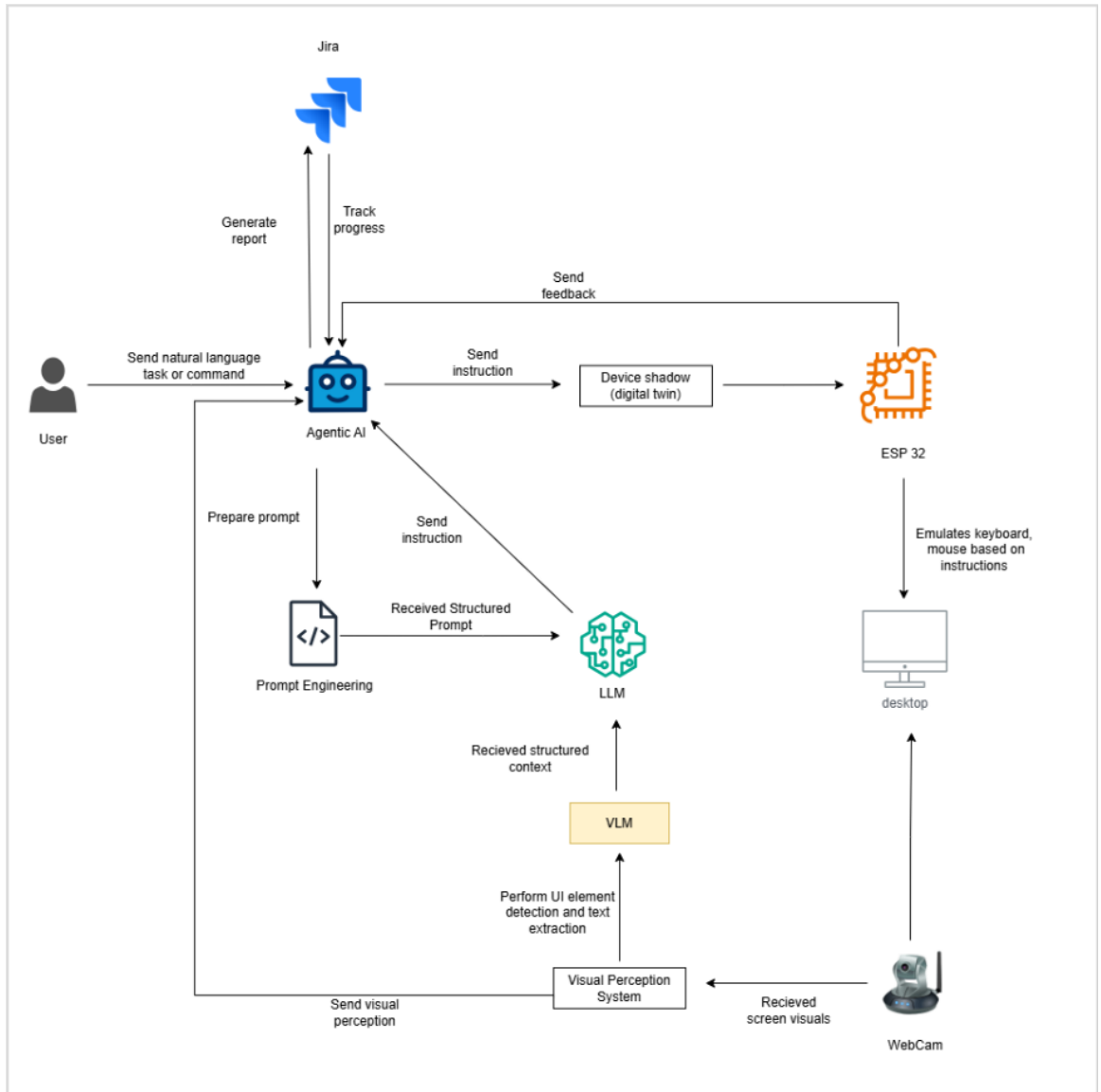


Figure 4.1: Overall system architecture diagram

Figure 4.1 presents the end-to-end operational flow of the proposed intelligent automation framework. The process begins when a user submits a natural language command to the Agentic AI controller. This command may include requests such as opening an application, logging into a system, generating reports, or navigating a dashboard.

After receiving the command, the Agentic AI coordinates two critical processes simultaneously. First, it prepares structured prompts for the language reasoning module. Second, it requests current screen context from the Visual Perception

System. A webcam continuously captures the target desktop screen and transmits live visual data to the perception subsystem.

The Visual Perception System analyzes the received screen image and identifies visible user interface components such as buttons, text fields, labels, tabs, menus, and icons. The extracted structured context is forwarded to the language reasoning module, enabling the system to understand the current interface state.

Based on the user objective and live screen context, the reasoning engine determines the next best action and returns executable instructions to the Agentic AI controller. These instructions are then transmitted to the ESP32-based Human Interface Device (HID) controller through the device shadow layer. The ESP32 emulates mouse movements, clicks, and keyboard inputs on the target computer exactly as a real human user would.

Once the action is performed, the screen changes are again captured through the webcam, thereby creating a continuous perception-action feedback loop. This closed-loop methodology allows the system to adapt dynamically to interface responses until task completion.

The architecture also supports enterprise integration features such as Jira progress tracking and automated report generation, demonstrating how intelligent screen automation can operate within real organizational workflows.

Summary of technologies, techniques, architecture, and algorithms used for effective resource utilization is shown in Table 4.1 below.

Component	Technology/ Method	Purpose
Frontend Interface	React + Vite	User interaction and task submission
Backend Orchestrator	FastAPI	Agent control and service coordination
Visual Perception	Gemini VLM	Semantic screen understanding

Image Processing	OpenCV	Frame preprocessing and enhancement
OCR Support	Tesseract / EasyOCR / PaddleOCR	Text extraction
Data Handling	NumPy / JSON	Structured output processing
Hardware Control	ESP32	Keyboard and mouse emulation
Communication	REST APIs / Serial HID	Inter-service and device communication
Validation	Schema + Coordinate Checks	Reliable perception output
Previous Models	YOLOv8 / Ollama VLM	Prototype evaluation

Table 4.1: Technologies and Techniques Used in the Proposed System

To further represent modular deployment, a layered architectural model was developed. **Figure 4.2** presents the high-level layered architecture of the system.

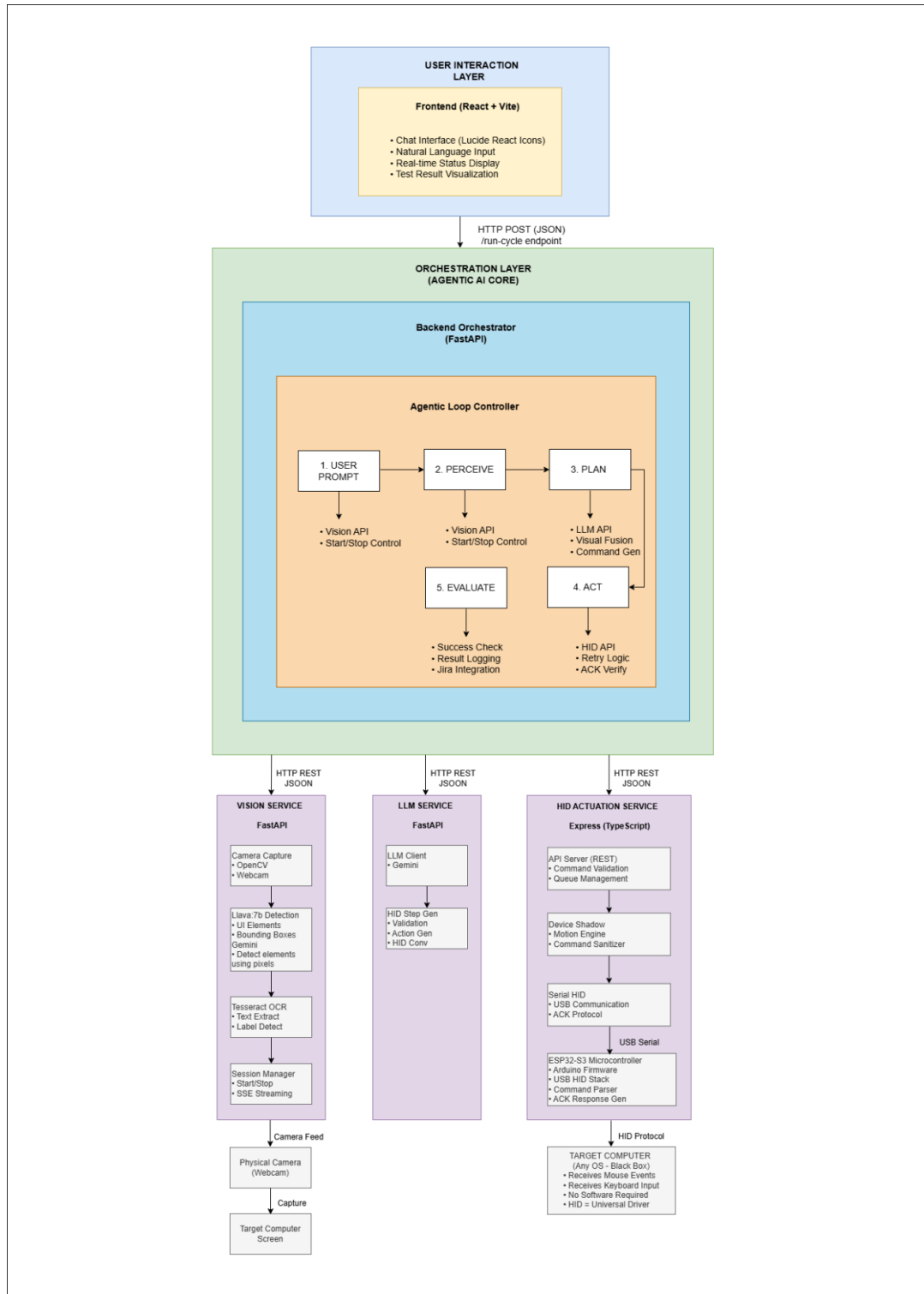


Figure 4.2: High-level layered architecture diagram

4.2 Specific Component Description

The Visual Perception Component is the core sensory subsystem of the proposed Screen-Aware Task Agent. Its primary responsibility is to observe the target screen, interpret visible interface elements, and convert raw visual data into structured machine-readable outputs that can be used by the planning and execution layers. Since the entire automation workflow depends on accurate understanding of the current screen state, the effectiveness of the complete system is strongly influenced by the performance of this component.

The component was developed as an independent FastAPI-based microservice to ensure modularity, scalability, and ease of maintenance. It receives screen images through webcam capture, processes them through a multimodal perception pipeline, validates outputs, and returns structured JSON responses. The final production implementation uses Gemini as the primary Vision-Language Model due to its strong semantic reasoning and generalized interface understanding capabilities.

The following subsections explain the purpose, workflow, internal modules, and operational methodology of the Visual Perception Component.

4.2.1 Purpose

The primary purpose of the Visual Perception Component is to provide real-time screen awareness to the intelligent automation system. Unlike traditional automation tools that depend on DOM selectors, APIs, accessibility metadata, or internal application structures, the proposed component operates entirely through visual observation. This enables automation in black-box environments where no software-level integration is available.

The component is designed to detect and understand visible user interface elements such as buttons, input fields, menus, labels, icons, tabs, browser controls, side panels, and other actionable screen objects. It also extracts positional information including pixel coordinates and bounding boxes, allowing downstream modules to determine where actions such as clicks or typing should occur.

Another important purpose of the component is to provide contextual understanding rather than simple object recognition. For example, it should distinguish whether a visible element is a clickable button, disabled control, text label, or form field. This semantic understanding significantly improves decision-making accuracy within the automation loop.

The module also supports transparency and maintainability by generating debug overlays, structured logs, and validation reports. Therefore, the purpose of the component extends beyond detection alone and includes enabling safe, interpretable, and scalable intelligent automation.

4.2.2 Workflow Overview

The Visual Perception Component follows a multi-stage processing workflow that transforms raw screen imagery into validated structured outputs. The workflow begins when the orchestration engine requests the current state of the target interface. A webcam source captures the latest screen frame and forwards it to the perception service.

The received image first enters the preprocessing stage, where image enhancement techniques are applied to improve quality and standardize input conditions. After preprocessing, the optimized frame is submitted to the Gemini Vision-Language Model for semantic UI analysis.

Gemini examines the screen and identifies visible components such as buttons, text fields, menus, labels, tabs, and icons. It also interprets contextual meaning and returns structured predictions.

The raw output is then parsed into JSON objects containing UI labels, element types, confidence scores, bounding boxes, and coordinates. A validation stage checks whether outputs are logically consistent and usable for automation execution.

Finally, the validated results are returned to the Agentic AI controller, which uses the information for planning and physical interaction with the target computer.

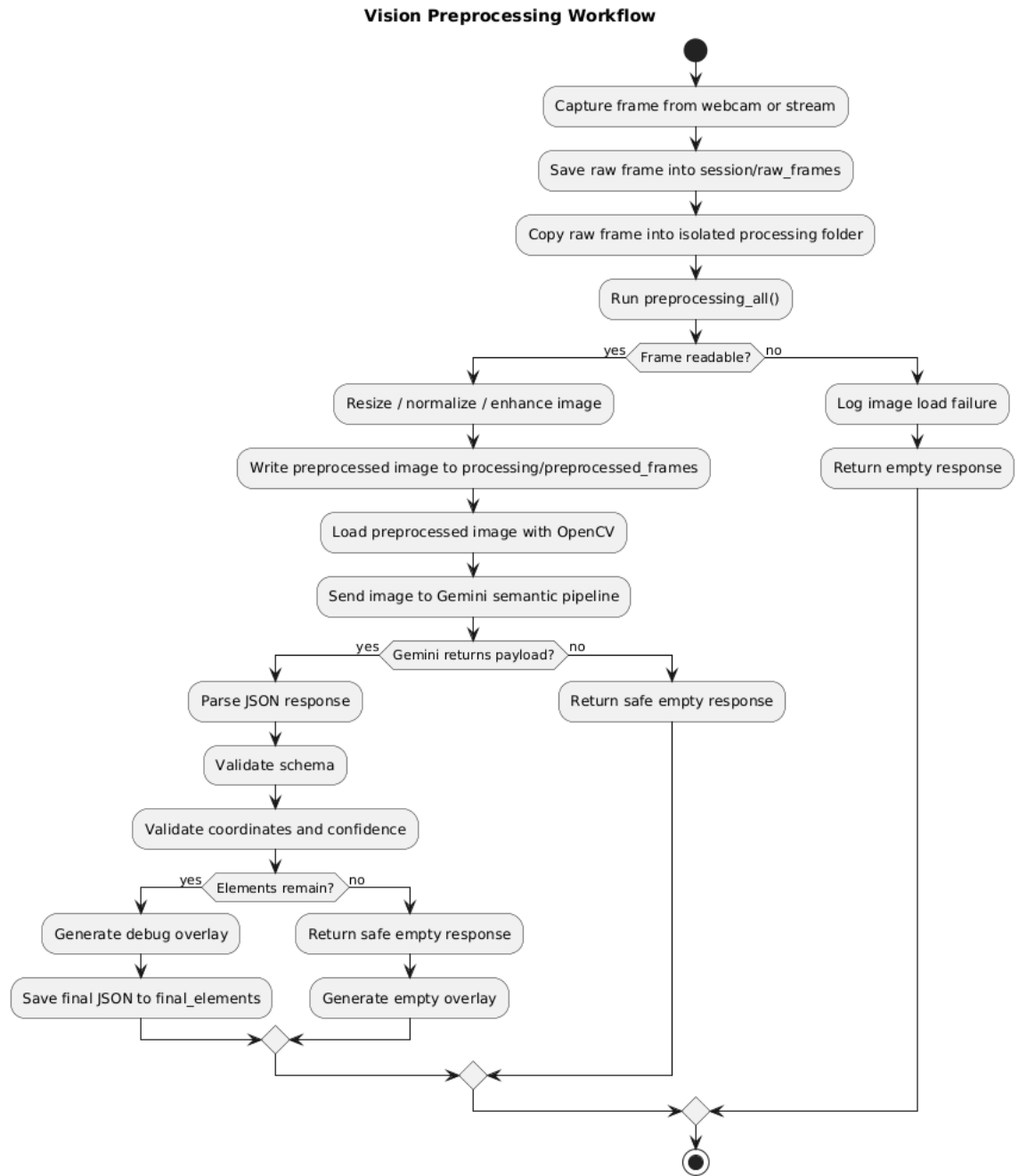


Figure 4.3: Vision processing workflow

4.2.3 Input Acquisition Stage

The first operational stage of the component is image acquisition. Accurate perception depends on receiving clear and up-to-date visual information from the target environment.

In the proposed system, a webcam is positioned to observe the target monitor externally. This allows black-box automation without installing any software on the controlled computer.

Each captured frame is associated with a session context and timestamp. This enables continuous monitoring and traceability across multiple automation cycles. Since the system operates iteratively, new frames may be requested after every executed action.

Reliable acquisition is essential because outdated or low-quality frames can lead to incorrect perception decisions.

4.2.4 Image Preprocessing Stage

Before AI analysis, the captured image is passed through a preprocessing pipeline implemented using OpenCV. The purpose of preprocessing is to improve image clarity, normalize input quality, and reduce visual noise that may negatively affect model performance.

Typical preprocessing operations include:

- Image resizing to supported dimensions
- Cropping irrelevant borders or empty regions
- Brightness and contrast enhancement
- Noise reduction and smoothing
- Color normalization
- Overlay preparation for debugging outputs

These steps help standardize frames captured under different lighting conditions, resolutions, and camera positions. Improved image quality leads to more accurate UI detection and semantic understanding.

4.2.5 Semantic UI Detection Stage

The semantic detection stage is the core intelligence layer of the component. In the final production implementation, Gemini is used as the primary Vision-Language Model.

Unlike conventional object detectors that recognize only predefined classes, Gemini analyzes the screen holistically and reasons about the functional meaning of visible regions. It can identify common interface elements such as:

- Buttons
- Input fields
- Labels
- Menus
- Tabs
- Hyperlinks
- Icons
- Sidebars
- Browser controls
- Navigation panels
- Taskbar items

For example, the model can infer that a rectangular region containing the text “Login” likely represents a button, while an empty box beside the word “Password” likely represents an input field.

This generalized understanding enables the system to perform effectively even on previously unseen interfaces.

4.2.6 Structured Output Generation

After semantic analysis, the raw model response is converted into deterministic structured outputs that can be consumed by software modules.

Each detected element is transformed into a JSON object containing fields such as:

- Element type
- Label or name
- Description
- Confidence score

- Pixel coordinates (dx, dy)
- Bounding box dimensions
- State information (if available)

This structure allows the planning engine to reason computationally rather than relying on natural language descriptions. For example, if the next step is to click a login button, the controller can directly use returned coordinates.

Structured output generation therefore bridges the gap between AI perception and executable automation.

4.2.7 Validation and Reliability Stage

Because automation actions depend directly on perception accuracy, validation is an essential stage of the workflow.

The component includes internal validators that inspect outputs before they are released to the orchestration engine. Validation checks include:

- JSON schema correctness
- Missing mandatory fields
- Invalid coordinates outside screen bounds
- Overlapping duplicate detections
- Extremely low-confidence predictions
- Corrupted bounding box values

If issues are detected, outputs may be corrected, filtered, or flagged for retry. This improves operational reliability and reduces the risk of incorrect automation actions.

4.2.8 Debugging and Session Management

To support maintainability and performance monitoring, the component generates session artifacts during runtime.

These may include:

- Original captured frame
- Preprocessed image
- Detection overlay image
- JSON response logs
- Timing information
- Error reports

Session data enables developers to inspect failures, evaluate model behavior, and continuously improve the perception pipeline. It is also useful for academic experimentation and benchmark reporting.

4.2.9 Previous Prototype Approaches

During earlier development stages, multiple alternative perception approaches were explored before finalizing the Gemini-based architecture.

These included:

- YOLOv8 for fast UI element localization
- Local Vision-Language Models through Ollama
- OCR-assisted pipelines using Tesseract, EasyOCR, and PaddleOCR
- Hybrid YOLO + VLM workflows

Although these approaches provided useful results, they showed limitations in semantic understanding and generalization across diverse software interfaces. Therefore, the final production system adopted Gemini as the primary perception engine.

4.2.10 Final Operational Outcome

The final Visual Perception Component provides a practical perception service capable of observing screens, understanding interface layouts, identifying actionable elements, validating outputs, and supplying structured data for downstream automation.

As a result, the component functions as the “eyes” of the Screen-Aware Task Agent and plays a critical role in enabling intelligent real-world GUI automation in black-box environments.

4.3 Design System Diagrams

System design diagrams are important tools used to visually represent the structure, data movement, module relationships, and operational logic of the proposed solution. In this research, design diagrams are used to clearly communicate how the Visual Perception Component interacts with other modules of the Screen-Aware Task Agent and how visual data is transformed into actionable outputs.

Since the proposed system combines computer vision, multimodal reasoning, backend APIs, and hardware actuation, textual explanations alone may not fully illustrate the internal workflow. Therefore, system diagrams improve clarity, academic presentation quality, and reproducibility of the implemented methodology.

For the Visual Perception Component, the most relevant diagrams include the Data Flow Diagram (DFD), Component Diagram, Sequence Diagram, and Process Flow Diagram. Among these, the DFD is highly useful because it focuses on movement of data between system entities and internal processes.

4.3.1 DFD Diagram

The Data Flow Diagram illustrates how visual data moves through the perception system. Screen frames are captured as input, passed through preprocessing, forwarded to the Gemini multimodal reasoning engine, and the structured output is returned to the orchestration layer for downstream action planning.

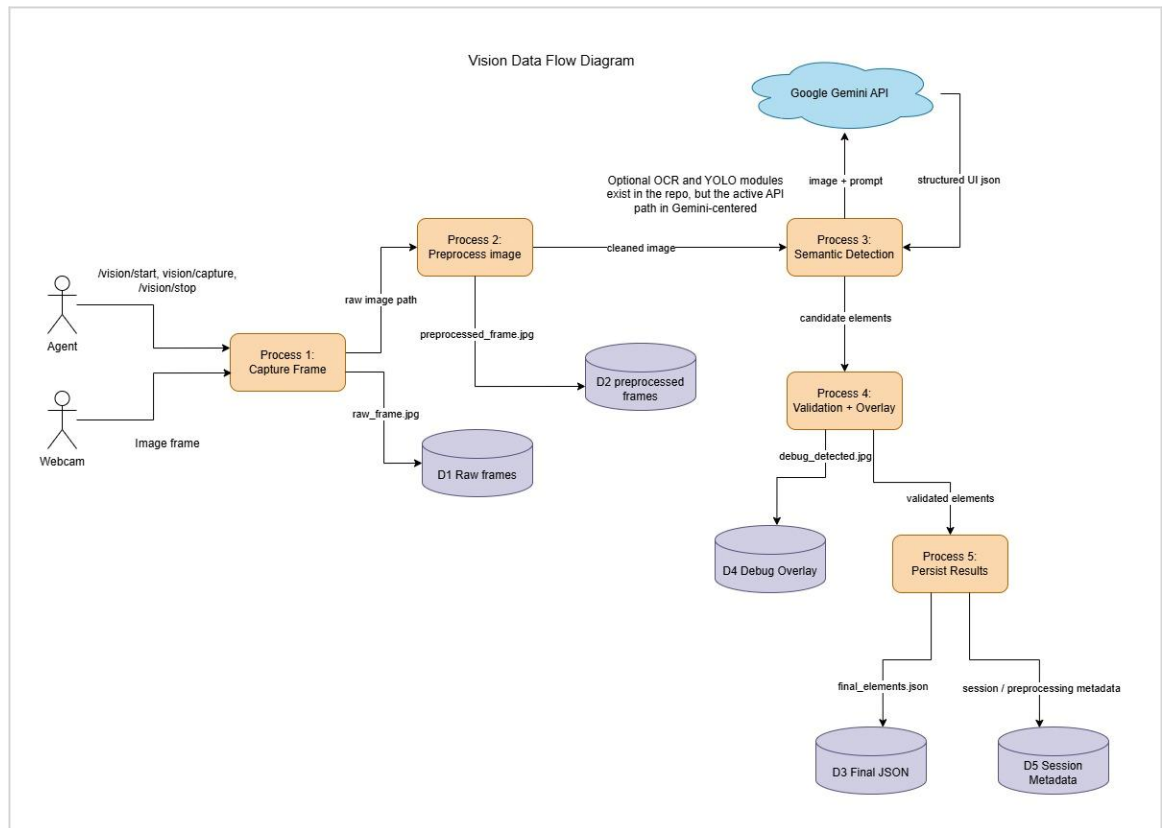


Figure 4.4: Visual perception data flow diagram

5. IMPLEMENTATION AND TESTING

5.1 Implementation

The implementation was carried out using Python-based technologies including FastAPI, OpenCV, NumPy, and Google Gemini Vision capabilities. The final architecture was selected after evaluating earlier prototype approaches involving YOLOv8 and locally hosted Vision-Language Models. The production version emphasizes generalized UI understanding, modular service integration, and reliable JSON-based outputs suitable for real-world intelligent automation systems.

5.1.1 Vision Service Overview

The Visual Perception Component was implemented as an independent FastAPI microservice to support modular deployment and clean separation from the orchestration and execution layers. The service was designed to expose REST-based endpoints through which other system components can submit screen images and receive structured interface understanding results.

The service architecture follows a request-response model. When the Agentic AI controller requires awareness of the current screen state, it sends an image request to the Vision Service. The service then processes the image through preprocessing, multimodal analysis, validation, and response generation stages before returning structured JSON outputs.

The use of FastAPI provides several advantages including lightweight deployment, asynchronous request handling, automatic API documentation, strong Python ecosystem compatibility, and simplified integration with AI inference pipelines. The service can be executed locally or deployed on a dedicated server depending on performance and scalability requirements.

The implementation was organized into modular source files including API routing, image preprocessing utilities, Gemini backend integration, OCR support modules, validation layers, and session logging mechanisms. This modular design improves maintainability and future extensibility.

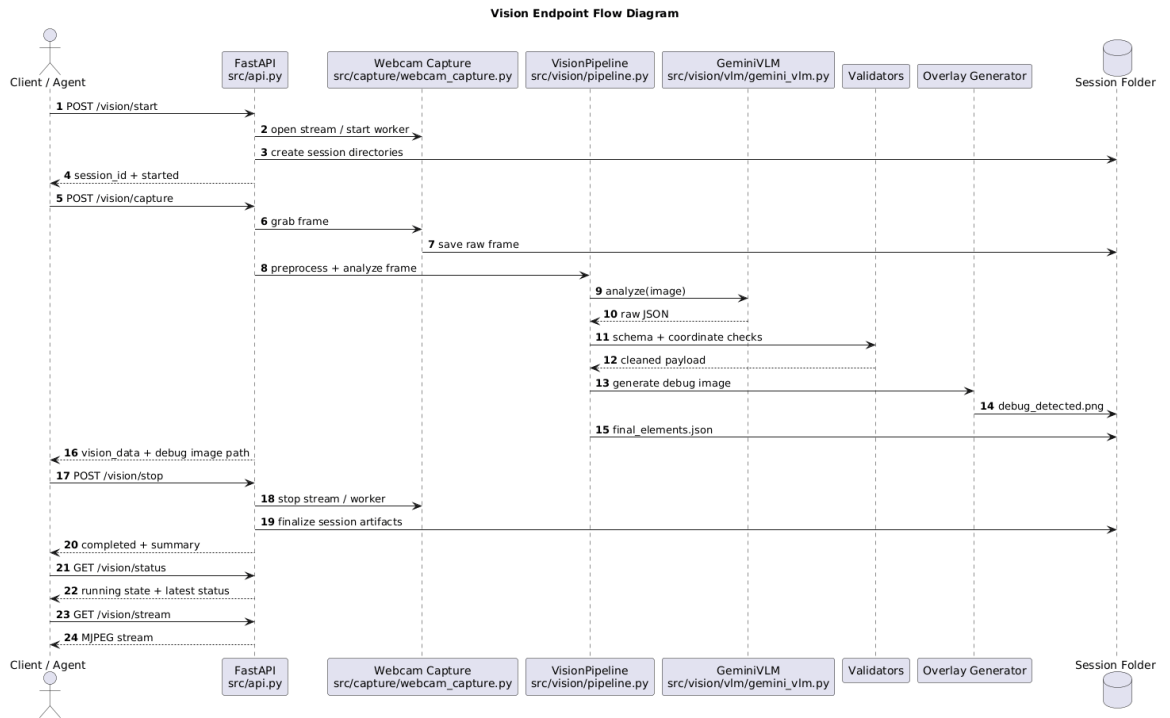


Figure 5.1: Vision endpoint flow diagram

5.1.2 Gemini Backend Integration

The final production implementation uses Google Gemini as the primary multimodal perception engine. Gemini was selected because it demonstrated stronger semantic reasoning, improved understanding of unseen graphical interfaces, and more reliable structured outputs when compared with earlier prototype models.

The Vision Service establishes secure communication with Gemini through API credentials loaded with environment configuration files using python-dotenv. This ensures separation between source code and sensitive credentials while supporting maintainable deployment practices.

During runtime, preprocessed images are forwarded to the Gemini backend together with carefully designed prompts requesting detection of visible UI components such as buttons, input fields, tabs, menus, labels, icons, and other relevant screen objects. The model returns structured textual outputs describing detected elements and their approximate locations.

These responses are then parsed into machine-readable JSON structures by the Vision Service. The integration methodology allows the system to benefit from advanced multimodal reasoning without embedding heavy models directly into the local runtime environment.

Earlier experiments with YOLOv8 and locally hosted models through Ollama demonstrated faster inference in constrained scenarios. However, Gemini provided superior generalization across unknown interfaces and therefore became the preferred production solution.

```
21 class GeminiVLM(BaseVLM):
147     def _system_prompt(image_width: int, image_height: int) -> str:
148         return (
149             "You are a UI detector for automation. Output ONLY valid JSON. "
150             "Do not output markdown. Do not output explanations. "
151             "Do not use double quotes inside label or description text. "
152             "Keep descriptions short, factual, and free of line breaks. "
153             "Detect every visible screen and application UI element that helps understand the screen, "
154             "including buttons, inputs, links, tabs, menus, checkboxes, radios, dropdowns, toggles, "
155             "panes, taskbar items, toolbar controls, browser chrome, window chrome, sidebar entries, "
156             "headings, labels, icons, status bars, and meaningful text. "
157             "Prefer high recall over minimalism. If the screen is complex, return many small elements. "
158             "Do not collapse the whole screen into only a few detections. "
159             "Return this exact root shape with required fields: "
160             "{image, image_size, coordinate_system, element_count, elements}. "
161             "coordinate_system must be 'pixel'. "
162             "elements must be an array of objects with fields: "
163             "id, type, label, description, state, dx, dy, confidence, source, bbox. "
164             "confidence must be numeric in [0,1]. "
165             "bbox must be a tight pixel box [x_min, y_min, x_max, y_max] around the element. "
166             f"dx must be integer in [0,{image_width}]. "
167             f"dy must be integer in [0,{image_height}]. "
168             "source must be 'gemini_vlm'. "
169             "element_count must equal elements array length. "
170             "When uncertain, prefer a smaller accurate element over a wrong one, but do not omit visible UI "
171             "that is clearly present. "
172             "If no interactive elements are present, return element_count=0 and elements=[]."
173         )
174
175     @staticmethod
176     def _broader_system_prompt(image_width: int, image_height: int) -> str:
177         return (
178             "You are a screen understanding model for desktop automation. Output ONLY valid JSON. "
179             "Do not use double quotes inside label or description text. "
180             "Keep descriptions short, factual, and free of line breaks. "
181             "Detect any visible UI structure that helps understand the screen, including tabs, panes, menus, "
182             "buttons, inputs, taskbar items, terminal controls, status bars, labels, icons, browser chrome, "
183             "window chrome, sidebars, headings, and text. "
184             "Prefer exhaustive reporting. Return many elements if the screen is dense. "
185             "Return this exact root shape with required fields: "
186             "{image, image_size, coordinate_system, element_count, elements}. "
187             "coordinate_system must be 'pixel'. "
188             "elements must be an array of objects with fields: "
189             "id, type, label, description, state, dx, dy, confidence, source, bbox. "
190             "confidence must be numeric in [0,1]. "
191             "bbox must be a tight pixel box [x_min, y_min, x_max, y_max] around the element. "
192             f"dx must be integer in [0,{image_width}]. "
193             f"dy must be integer in [0,{image_height}]. "
```

Figure 5.2: Gemini client implementation flow

```

vision > src > vision > vlm > gemini_vlm.py > ...
21  E:\slit-project\ScreenAwareTaskAgent\vision > Contains emphasized items
444
445
446     def _parse_json_payload(
447         self,
448         raw_text: str,
449         image_path: str,
450         image_width: int,
451         image_height: int,
452     ) -> Dict[str, Any]:
453         if not raw_text:
454             logger.warning("Gemini returned empty response content")
455             return self._safe_empty_response(
456                 image_path, image_width, image_height, vlm_error_type="parse_error"
457             )
458
459         candidate = raw_text.strip()
460         logger.debug("Raw Gemini response (trimmed): %s", candidate[:2000])
461         if candidate.startswith("```"):
462             candidate = candidate.replace("```json", "").replace("```", "").strip()
463
464         parsed: Any
465         try:
466             parsed = json.loads(candidate)
467         except json.JSONDecodeError:
468             extracted = self._extract_best_json_object(candidate)
469             extracted_array = candidate if candidate.lstrip().startswith("[") else ""
470             if not extracted and not extracted_array:
471                 recovered = self._recover_partial_payload(candidate, image_path, image_width, image_height)
472                 if recovered is not None:
473                     return recovered
474                 try:
475                     debug_path = f"{image_path}.gemini_raw.txt"
476                     with open(debug_path, "w", encoding="utf-8") as fh:
477                         fh.write(candidate[:20000])
478                     logger.warning("Wrote raw Gemini response to %s", debug_path)
479                 except Exception:
480                     logger.exception("Failed writing raw Gemini response to disk")
481                     logger.exception("No JSON object found in Gemini response")
482             return self._safe_empty_response(
483                 image_path, image_width, image_height, vlm_error_type="parse_error"
484             )
485
486         parse_candidates = [c for c in (extracted, extracted_array) if c]
487         parsed = None
488         for text in parse_candidates:
489             for variant in (
490                 text,

```

Figure 5.3: Gemini client implementation

```

vision > src > vision > config.py > ...
1  """Configuration for Gemini-backed vision pipeline."""
2
3  from __future__ import annotations
4
5  import os
6
7  MODEL_NAME = os.getenv("GEMINI_MODEL_NAME", "gemini-2.0-flash-lite")
8  CONFIDENCE_THRESHOLD = 0.65
9
10 # GEMINI API key from environment
11 GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
12 # Timeout (seconds) for Gemini RPCs. Can be overridden in vision/.env
13 try:
14     GEMINI_TIMEOUT_SECONDS = float(os.getenv("GEMINI_TIMEOUT_SECONDS", "90"))
15 except Exception:
16     GEMINI_TIMEOUT_SECONDS = 90.0
17
18 # Whether to attempt automatic screen boundary detection and crop dark borders.
19 # Set to "0" in environment to disable cropping and always use full-frame images.
20 DETECT_SCREEN_BOUNDARIES = os.getenv("VISION_DETECT_BOUNDARIES", "0") != "0"
21

```

Figure 5.4: Gemini API configuration

```

vision > src > perception > vlm > prompt_templates.py > ...
6 from typing import Optional
7
8 UI_DISCOVERY_PROMPT = """
9 You are a high-recall, high-precision UI parser for automation.
10
11 Goal:
12 Detect every visible UI element on the screen, not just the obvious ones:
13 buttons, input fields, labels, static text, table cells, tabs, menus, icons,
14 cards, list items, chips, links, toggles, checkboxes, radios, dialogs, images,
15 browser chrome, window chrome, status bars, sidebars, toolbars, headings, and
16 other meaningful on-screen controls or text.
17
18 Hard constraints:
19 1. Return ONLY a valid JSON object (no markdown, no commentary).
20 2. Detect only what is visible in the screenshot.
21 3. Bounding boxes must be tight and axis-aligned around each individual element.
22 4. Use bbox = [x_min, y_min, x_max, y_max], with x_min < x_max and y_min < y_max.
23 5. Coordinates can be normalized (0-1) or pixels; stay consistent.
24 6. Never leave label empty: if no visible text, create a short functional label
25    | like "search icon", "profile avatar", "submit button", "data row 3".
26 7. Never leave description empty: one concrete sentence about what the element
27    | shows or does in context.
28 8. Use type "unknown" only as last resort.
29 9. Keep confidence realistic (0.0-1.0).
30 10. Prefer recall over minimalism. If the screen is complex, return many small
31     | elements instead of collapsing them into only a few large ones.
32
33 Allowed types:
34 button, input_field, text, label, icon, dropdown, checkbox, radio, menu, tab,
35 modal, dialog, link, card, list_item, image, unknown
36
37 Output schema:
38 {
39   "elements": [
40     {
41       "id": "element_1",
42       "type": "button",
43       "label": "Save",
44       "description": "Primary action button to persist the current form values.",
45       "state": "enabled",
46       "bbox": [0.35, 0.70, 0.65, 0.85],
47       "confidence": 0.95
48     }
49   ],
50   "page_structure": {
51     "title": "short title",
52     "layout": "brief layout description",
53     "density": "sparse|normal|dense"
54   }
55 }

```

Figure 5.5: Prompt used for UI detection

5.1.3 Image Input and Preprocessing Pipeline

Before AI analysis can occur, incoming images must be standardized and enhanced through preprocessing. The Vision Service was designed to accept inputs from multiple sources including webcam captures.

In the black-box operational environment, a webcam is positioned externally to observe the target monitor. This approach allows automation without installing software on the controlled machine.

Once received, images pass through a preprocessing pipeline implemented using OpenCV. Typical operations include resizing, cropping, normalization, denoising, and contrast enhancement. These operations reduce sensitivity to lighting variations, image compression noise, camera angle inconsistencies, and screen resolution differences.

Preprocessing is especially important because poor image quality can significantly degrade multimodal reasoning performance. By standardizing inputs before Gemini inference, the system improves consistency and overall detection reliability.

Image Preprocessing Pipeline

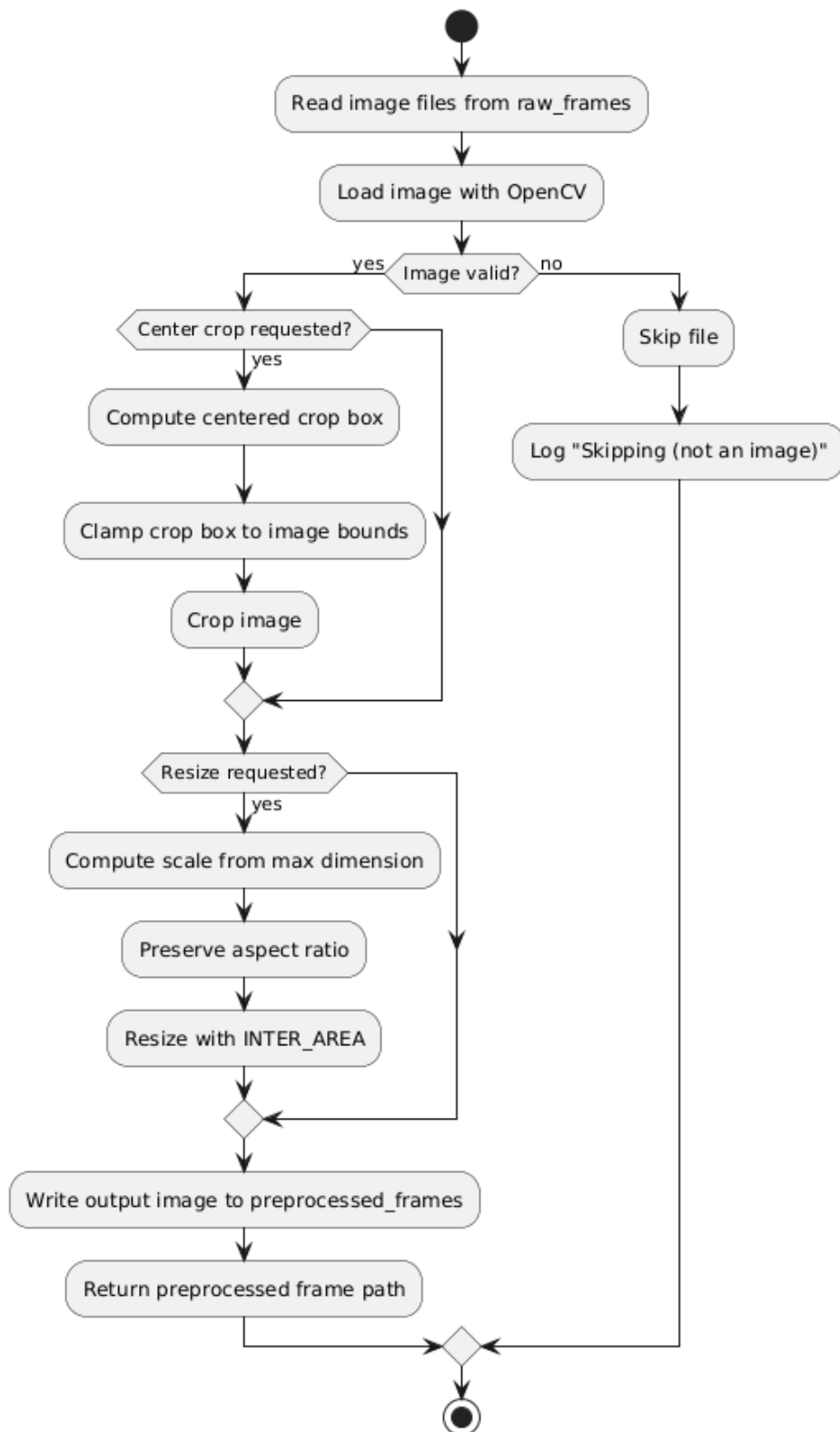


Figure 5.6: Image Preprocessing Pipeline

```

vision > src > preprocessing > preprocess.py > ...
19
20
21 def preprocess_all(raw_dir=None, out_dir=None, crop=None, resize=None, ext_in=".jpg", ".png")
22     """Preprocess all images found in raw_dir and write to out_dir.
23
24     If raw_dir or out_dir are None, defaults inside data/ are used.
25     """
26     raw_dir = raw_dir or DEFAULT_RAW_DIR
27     out_dir = out_dir or DEFAULT_OUT_DIR
28     os.makedirs(out_dir, exist_ok=True)
29
30     files = [f for f in sorted(os.listdir(raw_dir)) if f.lower().endswith(ext_in)]
31     for fname in files:
32         path = os.path.join(raw_dir, fname)
33         img = cv2.imread(path)
34         if img is None:
35             print("Skipping (not an image):", path)
36             continue
37
38         if crop:
39             crop_w, crop_h = crop # width, height
40             H, W = img.shape[:2]
41
42             # compute centered crop
43             left = (W - crop_w) // 2
44             top = (H - crop_h) // 2
45
46             # ensure boundaries are valid
47             left = max(left, 0)
48             top = max(top, 0)
49             right = min(left + crop_w, W)
50             bottom = min(top + crop_h, H)
51
52             img = img[top:bottom, left:right]
53
54         if resize:
55             h, w = img.shape[:2]
56             # preserve aspect ratio: set max dim to resize
57             scale = resize / max(h, w)
58             new_w, new_h = int(w * scale), int(h * scale)
59             img = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_AREA)
60
61         out_path = os.path.join(out_dir, fname)
62         cv2.imwrite(out_path, img)
63         print("Wrote:", out_path)
64
65     if __name__ == "__main__":
66         parser = argparse.ArgumentParser()
67         parser.add_argument("--crop", nargs=2, type=int, help="center crop: width height")

```

Figure 5.7: OpenCV processing

5.1.4 UI Detection and Structured Output Generation

After preprocessing, the image is submitted to the Gemini Vision model for semantic analysis. Unlike traditional object detectors that only recognize predefined classes, Gemini interprets the visual scene and reasons about functional UI meaning.

The model identifies interface elements such as buttons, input fields, menus, links, tabs, labels, browser controls, taskbar objects, and icons. It can also infer likely interactivity from context. For example, a rectangular region containing the text “Login” may be

classified as a button, while an empty bordered region near “Username” may be identified as a text input field.

The raw model response is converted into structured JSON objects by the parsing layer. Each object typically contains:

- Element type
- Label or name
- Confidence score
- Pixel coordinates
- Bounding box dimensions
- Optional state information

This output format enables downstream planning modules to use perception results directly for action generation.

An example request and response payload when start the stream is shown below,

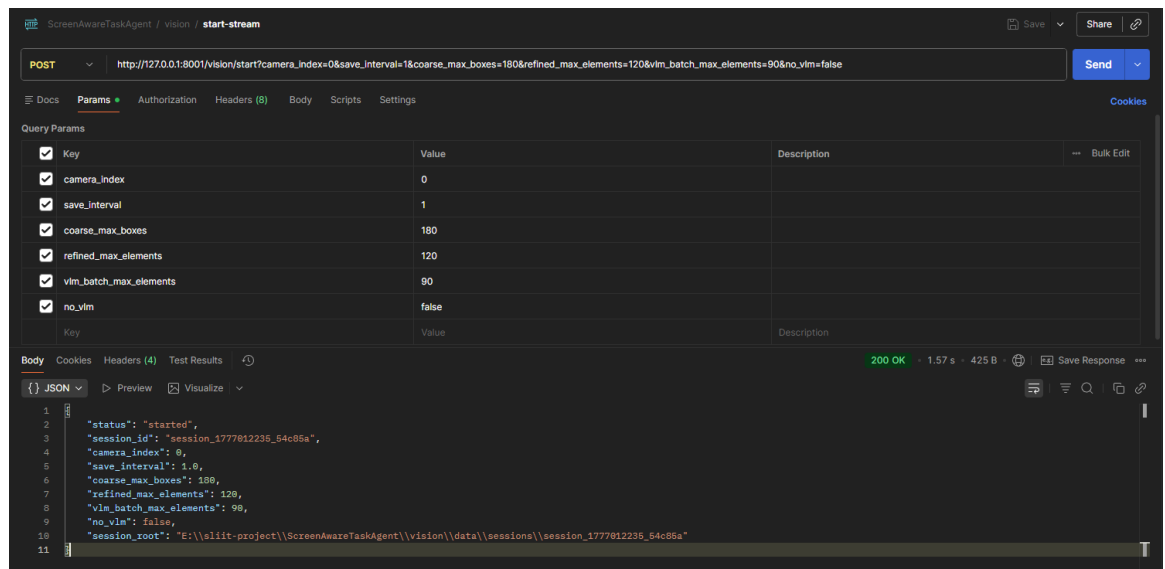
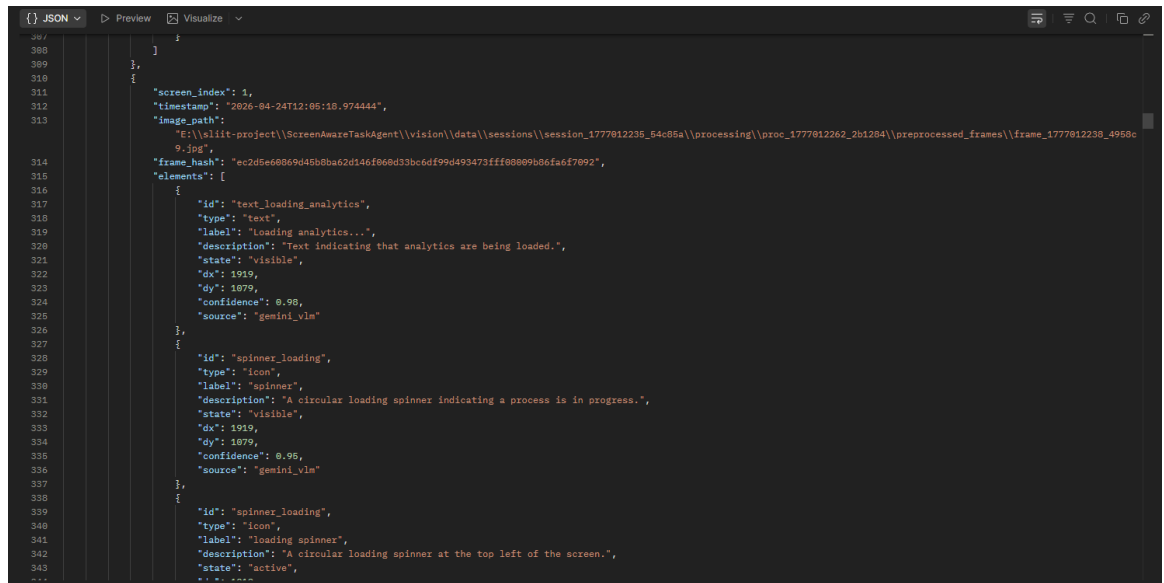


Figure 5.8: Sample request and response

An example response payload when stop the stream is shown below.

A screenshot of a code editor showing a JSON response payload. The editor has a dark theme and a sidebar on the left with line numbers from 307 to 343. The JSON object is a response from a stream, containing fields like 'screen_index', 'timestamp', 'image_path', 'frame_hash', and 'elements'. The 'elements' array contains three objects representing detected UI elements: 'text_loading_analytics', 'spinner_loading', and 'spinner_loading'. Each element object includes fields like 'type', 'label', 'description', 'state', 'dx', 'dy', 'confidence', and 'source'. The 'source' for all elements is 'gemini_vlm'.

```
{
  "screen_index": 1,
  "timestamp": "2026-04-24T12:05:18.974444",
  "image_path": "E:\\slit-project\\ScreenAwareTaskAgent\\vision\\data\\sessions\\session_1777812235_54c85a\\processing\\proc_1777812262_2b128d\\preprocessed_frames\\frame_1777812238_4958c9.jpg",
  "frame_hash": "ec2d5e68869d45b8b62d146f868d33bc6df99d493473fff88809b86fa6f7892",
  "elements": [
    {
      "id": "text_loading_analytics",
      "type": "text",
      "label": "loading analytics...",
      "description": "Text indicating that analytics are being loaded.",
      "state": "visible",
      "dx": 1919,
      "dy": 1879,
      "confidence": 0.98,
      "source": "gemini_vlm"
    },
    {
      "id": "spinner_loading",
      "type": "icon",
      "label": "spinner",
      "description": "A circular loading spinner indicating a process is in progress.",
      "state": "visible",
      "dx": 1919,
      "dy": 1879,
      "confidence": 0.96,
      "source": "gemini_vlm"
    },
    {
      "id": "spinner_loading",
      "type": "icon",
      "label": "loading spinner",
      "description": "A circular loading spinner at the top left of the screen.",
      "state": "active",
      "dx": 1919,
      "dy": 1879,
      "confidence": 0.96,
      "source": "gemini_vlm"
    }
  ]
}
```

Figure 5.9: Sample response payload

5.1.5 Validation, Coordinates, and Session Management

Because automation actions depend directly on perception accuracy, a validation layer was implemented to inspect model outputs before release to the orchestration engine.

The first validation stage checks schema correctness. Mandatory fields such as type, coordinates, and labels must exist in acceptable format. The second stage verifies coordinate validity to ensure points lie within visible screen boundaries. Bounding box values are checked for logical consistency. Duplicate or overlapping detections may also be filtered where necessary.

Confidence thresholds are used to reduce unreliable detections. If outputs fall below acceptable confidence, the system may reject or flag them for reanalysis.

The Vision Service also maintains session artifacts for debugging and traceability. These include original images, preprocessed images, structured JSON outputs, timing logs, and overlay images showing detected UI regions. Session management is valuable during

experimentation and production monitoring.

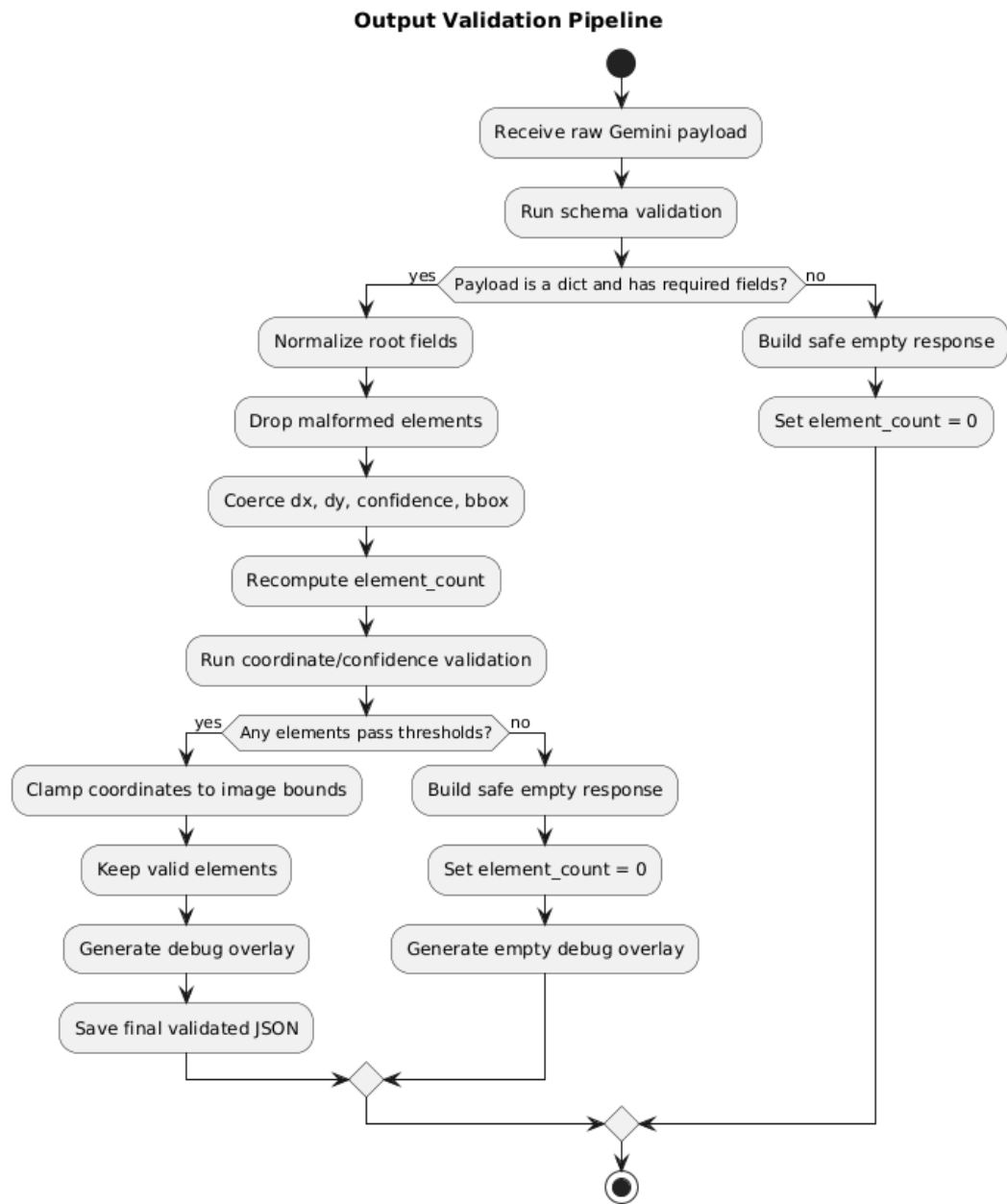


Figure 5.10: Output validation pipeline

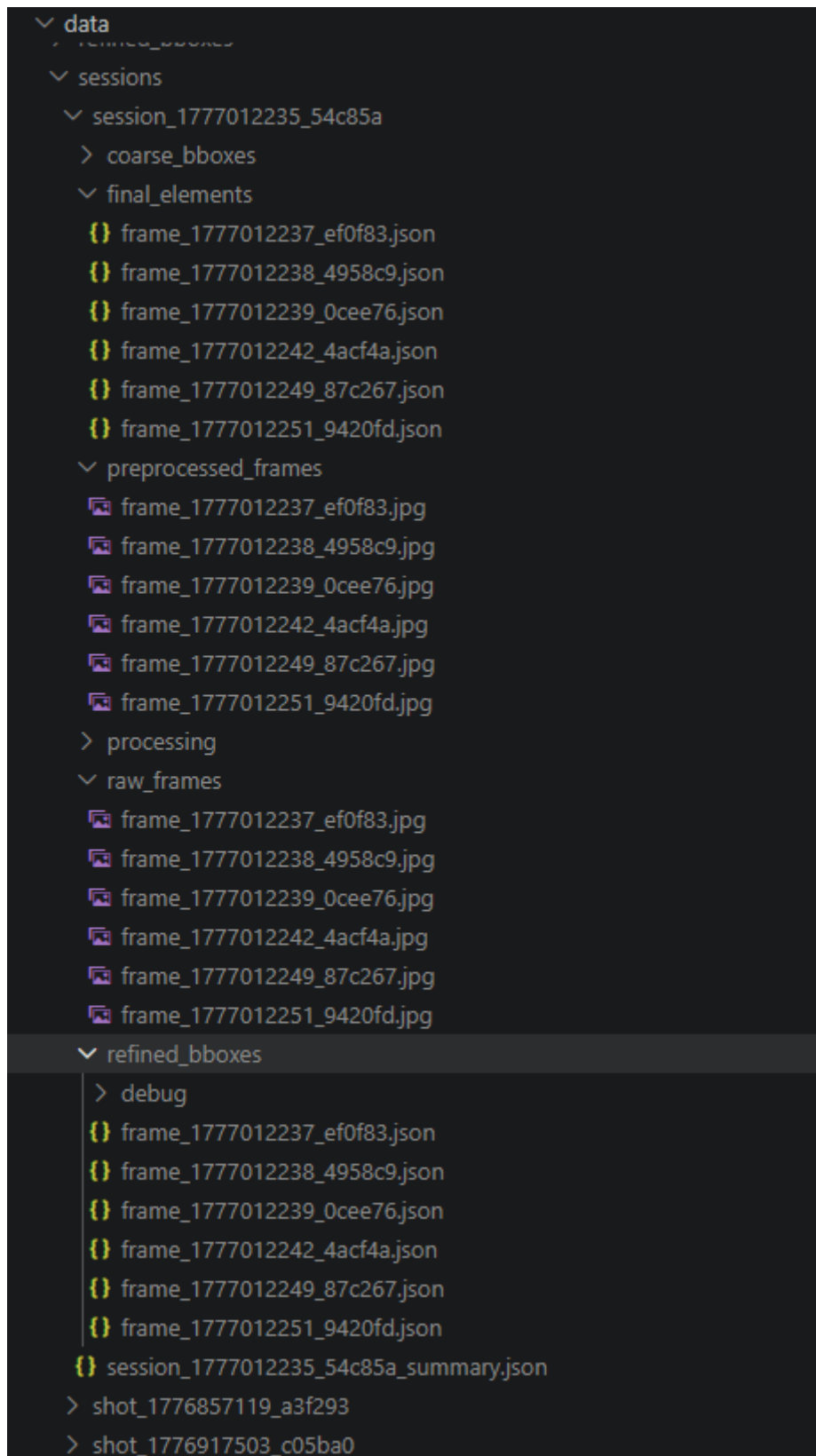


Figure 5.11: Snapshot of the session folder

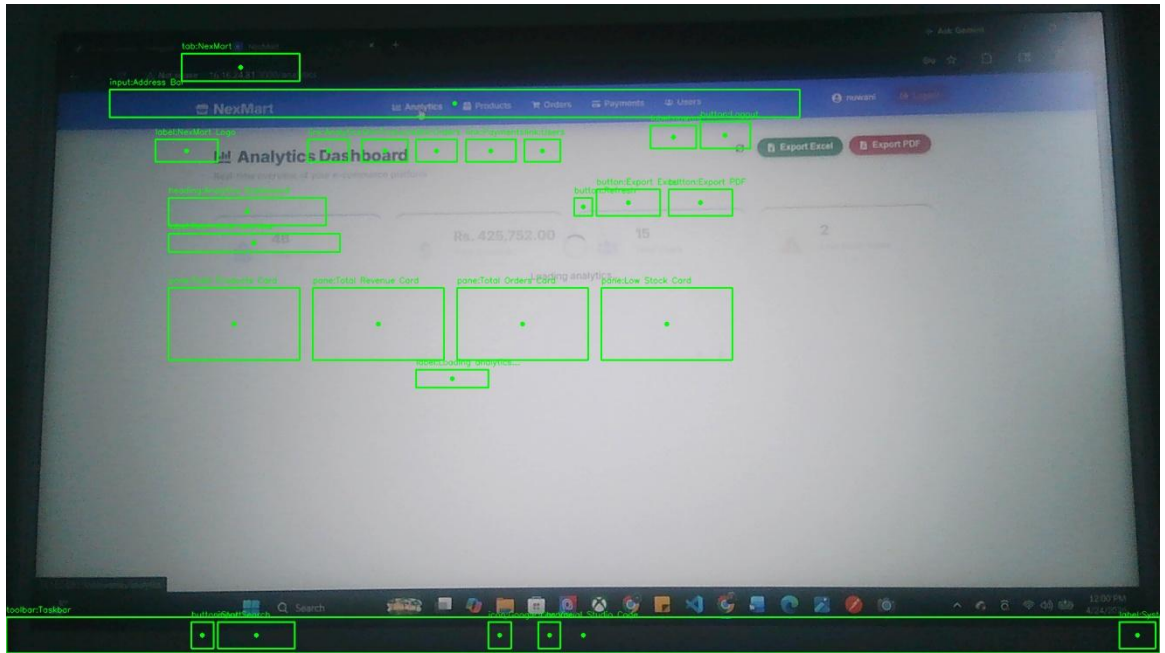


Figure 5.12: Debug overlay image

5.1.6 Model Selection and Operational Decision

During the development lifecycle, multiple perception strategies were evaluated before finalizing the production architecture.

Initial prototypes used YOLOv8 for rapid UI localization. YOLO demonstrated low latency and acceptable performance for known interface categories. Inference times were typically faster than cloud-based multimodal systems. However, YOLO required predefined classes and was less effective on unseen enterprise interfaces.

Additional experiments used locally hosted Vision-Language Models through Ollama. These approaches improved privacy and local control but showed limitations in reasoning quality and output consistency.

The final operational decision was to adopt Gemini as the primary production model because it offered:

- Stronger semantic screen understanding
- Better zero-shot generalization

- Improved detection of unseen UI layouts
- More reliable structured outputs
- Easier adaptation across software platforms

Comparative Evaluation of Perception Models

Model	Speed	Semantic Understanding	Generalization	Final Usage
YOLOv8	High	Moderate	Low	Prototype
Local VLM(Ollama)	Moderate	Moderate	Medium	Prototype
Gemini	Moderate	High	High	Production

Table 5.1: Summarizes comparative observation

The transition from specialized detectors to generalized multimodal perception represents a key engineering contribution of this research.

5.2 Testing

5.2.1 Test Plan and Strategy

A structured testing methodology was used to evaluate the functional correctness, performance, and operational reliability of the Vision Service.

Testing was divided into the following categories:

1. Functional Testing
Verifies whether the service correctly detects UI elements and returns structured outputs.
2. API Testing
Confirms endpoint accessibility, payload handling, and response formatting.
3. Performance Testing
Measures average response time under practical workloads.

4. Robustness Testing

Evaluates behavior under low-quality images, varied resolutions, and cluttered interfaces.

5. Integration Testing

Confirms compatibility with the Agentic AI controller and downstream modules.

Test images included login screens, dashboards, browsers, desktop menus, and application windows to ensure diverse coverage.

5.2.2 Test Case Design

The following test cases were designed to evaluate the reliability, functional correctness, output consistency, and real-world usability of the Visual Perception Component. Each test was conducted using representative screen images collected from practical desktop environments such as login pages, dashboards, browser windows, and application interfaces. The objective of these tests was to verify whether the system could accurately detect visible UI components, generate structured outputs, and maintain stable performance under different visual conditions.

Table 5.2: Verify login screen UI detection

Test Case Id	01
Test Case	Verify login screen UI detection
Test Scenario	Verify that the Vision Service correctly detects username field, password field, and login button from a standard login interface.
Input	Login screen image containing Username field, Password field, and Login button
Expected Output	1. Image should be processed successfully 2. UI elements should be detected with coordinates and confidence values. JSON response should be valid

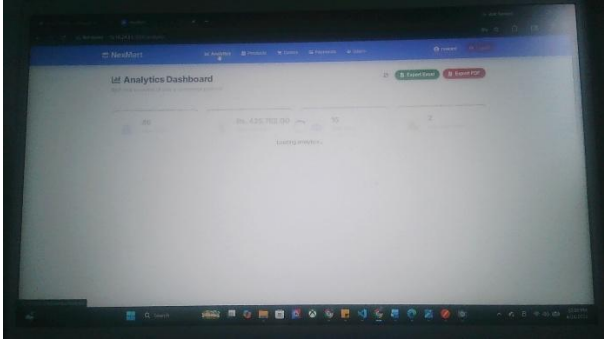
Actual Result	1. Processing completed successfully 2. Username input, Password input, and Login button were detected. Structured JSON response generated with confidence values above 0.90
Status (Pass/Fail)	Pass

Table 5.3: Verify missing UI element handling

Test Case Id	02
Test Case	Verify missing UI element handling
Test Scenario	Verify that the Vision Service correctly handles screens where expected elements are absent.
Input	Login page image containing Username and Password fields only (Login button not visible)
Expected Output	1. Available fields should be detected) 2. Missing Login button should not produce false detection 3. JSON output should remain valid
Actual Result	1. Username and Password fields detected successfully 2. No false Login button detection occurred. Valid JSON response returned
Status (Pass/Fail)	Pass

Table 5.4: Verify complex dashboard UI detection

Test Case Id	03
Test Case	Verify complex dashboard UI detection

Test Scenario	Verify that the Vision Service can detect multiple UI components in a cluttered dashboard interface.
Input	Dashboard screens containing side menu, search bar, buttons, cards, and top navigation 
Expected Output	1. Major visible UI elements should be identified 2. Coordinates should be returned correctly. Detection confidence should remain acceptable.
Actual Result	1. Sidebar, navigation bar, search field, and buttons detected successfully. 2. Coordinates returned accurately. Confidence range observed between 0.84–0.96

```

vision > data > sessions > session_1777012235_54c85a > final_elements > {} frame_1777012237_ef0f83.json > {} elements
1
2 {
3   "image": "E:\\slit-project\\ScreenAwareTaskAgent\\vision\\data\\sessions\\session_
4     "image_size": {
5       "width": 1920,
6       "height": 1080
7     },
8   "coordinate_system": "pixel",
9   "elements": [
10     {
11       "id": "browser_tab_nexmart",
12       "type": "tab",
13       "label": "NexMart",
14       "description": "Browser tab for the NexMart application",
15       "state": "active",
16       "dx": 1919,
17       "dy": 1079,
18       "confidence": 0.95,
19       "source": "gemini_vlm"
20     },
21     {
22       "id": "address_bar",
23       "type": "input",
24       "label": "Address Bar",
25       "description": "Browser address bar showing the URL 16.16.24.81:3000/analytics",
26       "state": "visible",
27       "dx": 1919,
28       "dy": 1079,
29       "confidence": 0.98,
30       "source": "gemini_vlm"
31     },
32     {
33       "id": "nexmart_logo",
34       "type": "label",
35       "label": "NexMart Logo",
36       "description": "NexMart application logo and name",
37       "state": "visible",
38       "dx": 1919,
39       "dy": 1079,
40       "confidence": 0.95,
41       "source": "gemini_vlm"
42     },
43     {
44       "id": "nav_analytics",
45       "type": "link",
46       "label": "Analytics",
47       "description": "Navigation link to the Analytics page",
48       "state": "active",
49       "dx": 1919,
50       "dy": 1079,
51       "confidence": 0.95,
52       "source": "gemini_vlm"
53     },
54     {
55       "id": "nav_products",
56       "type": "link",
57       "label": "Products",
58       "description": "Navigation link to the Products page",
59       "state": "visible",
60       "dx": 1919,
61       "dy": 1079,
62       "confidence": 0.95,
63       "source": "gemini_vlm"
64     },
65     {
66       "id": "nav_orders",
67       "type": "link",
68       "label": "Orders",
69       "description": "Navigation link to the Orders page",
70       "state": "visible",
71       "dx": 1919,
72       "dy": 1079,
73       "confidence": 0.95,
74       "source": "gemini_vlm"
75     },
76     {
77       "id": "nav_payments",
78       "type": "link",
79       "label": "Payments",
80       "description": "Navigation link to the Payments page",
81       "state": "visible",
82       "dx": 1919,
83       "dy": 1079,
84       "confidence": 0.95,
85       "source": "gemini_vlm"
86     },
87     {
88       "id": "nav_users",
89       "type": "link",
90       "label": "Users",
91       "description": "Navigation link to the Users page",
92       "state": "visible",
93       "dx": 1919,
94       "dy": 1079,
95       "confidence": 0.95,
96       "source": "gemini_vlm"
97     }
98   ]
99 }

```

```

8   "elements": [
42     {
43       "dx": 1919,
44       "dy": 1079,
45       "confidence": 0.95,
46       "source": "gemini_vlm"
47     },
48     {
49       "id": "nav_products",
50       "type": "link",
51       "label": "Products",
52       "description": "Navigation link to the Products page",
53       "state": "visible",
54       "dx": 1919,
55       "dy": 1079,
56       "confidence": 0.95,
57       "source": "gemini_vlm"
58     },
59     {
60       "id": "nav_orders",
61       "type": "link",
62       "label": "Orders",
63       "description": "Navigation link to the Orders page",
64       "state": "visible",
65       "dx": 1919,
66       "dy": 1079,
67       "confidence": 0.95,
68       "source": "gemini_vlm"
69     },
70     {
71       "id": "nav_payments",
72       "type": "link",
73       "label": "Payments",
74       "description": "Navigation link to the Payments page",
75       "state": "visible",
76       "dx": 1919,
77       "dy": 1079,
78       "confidence": 0.95,
79       "source": "gemini_vlm"
80     },
81     {
82       "id": "nav_users",
83       "type": "link",
84       "label": "Users",
85       "description": "Navigation link to the Users page",
86       "state": "visible",
87       "dx": 1919,
88       "dy": 1079,
89       "confidence": 0.95,
90       "source": "gemini_vlm"
91     }
92   ]
93 }

```

```

vision > data > sessions > session_1777012235_54c85a > final_elements > frame_1777012237_ef0f83.json > elements
8
"elements": [
86
{
90
  "description": "Navigation link to the Users page",
91
  "state": "visible",
92
  "dx": 1919,
93
  "dy": 1079,
94
  "confidence": 0.95,
95
  "source": "gemini_vlm"
96
},
97
{
98
  "id": "user_profile",
99
  "type": "label",
100
  "label": "nuwani",
101
  "description": "Logged in user profile name",
102
  "state": "visible",
103
  "dx": 1919,
104
  "dy": 1079,
105
  "confidence": 0.9,
106
  "source": "gemini_vlm"
107
},
108
{
109
  "id": "logout_button",
110
  "type": "button",
111
  "label": "Logout",
112
  "description": "Button to log out of the application",
113
  "state": "visible",
114
  "dx": 1919,
115
  "dy": 1079,
116
  "confidence": 0.9,
117
  "source": "gemini_vlm"
118
},
119
{
120
  "id": "dashboard_heading",
121
  "type": "heading",
122
  "label": "Analytics Dashboard",
123
  "description": "Main heading of the analytics page",
124
  "state": "visible",
125
  "dx": 1919,
126
  "dy": 1079,
127
  "confidence": 0.98,
128
  "source": "gemini_vlm"
129
},
130
{
131
  "id": "dashboard_subheading",
132
  "type": "label",
133
  "label": "Real-time overview",
134
  "description": "Subheading describing the dashboard content",
135
  "state": "visible",

```

```

vision > data > sessions > session_1777012235_54c85a > final_elements > frame_1777012237_ef0f83.json > elements
8
"elements": [
152
{
153
  "id": "export_excel_button",
154
  "type": "button",
155
  "label": "Export Excel",
156
  "description": "Button to export data to Excel",
157
  "state": "visible",
158
  "dx": 1919,
159
  "dy": 1079,
160
  "confidence": 0.95,
161
  "source": "gemini_vlm"
162
},
163
{
164
  "id": "export_pdf_button",
165
  "type": "button",
166
  "label": "Export PDF",
167
  "description": "Button to export data to PDF",
168
  "state": "visible",
169
  "dx": 1919,
170
  "dy": 1079,
171
  "confidence": 0.95,
172
  "source": "gemini_vlm"
173
},
174
{
175
  "id": "summary_card_products",
176
  "type": "pane",
177
  "label": "Total Products Card",
178
  "description": "Summary card showing 46 total products",
179
  "state": "visible",
180
  "dx": 1919,
181
  "dy": 1079,
182
  "confidence": 0.9,
183
  "source": "gemini_vlm"
184
},
185
{
186
  "id": "summary_card_revenue",
187
  "type": "pane",
188
  "label": "Total Revenue Card",
189
  "description": "Summary card showing Rs. 425,752.00 total revenue",
190
  "state": "visible",
191
  "dx": 1919,
192
  "dy": 1079,
193
  "confidence": 0.9,
194
  "source": "gemini_vlm"
195
},
196
{
197
  "id": "summary_card_orders",
198
  "type": "pane",

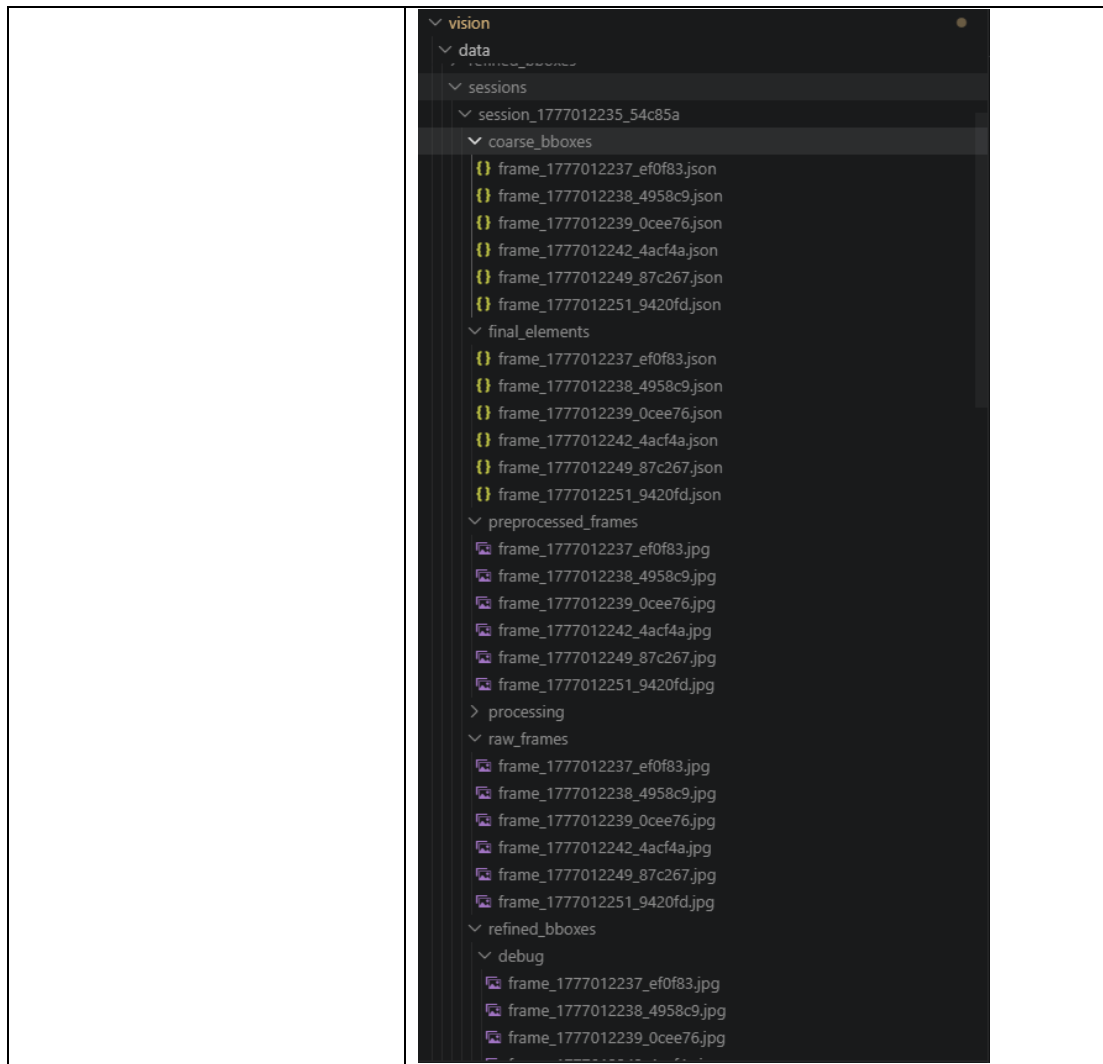
```


Expected Output	1. Response status should be success. 2. Detected elements should appear in structured JSON format 3. Coordinates and confidence fields should be included.
Actual Result	1. Status returned as success 2. JSON output contained 7 detected elements 3. All objects included coordinates, labels and confidence values
Status (Pass/Fail)	Pass

Table 5.6: Verify session artifact generation

Test Case Id	05
Test Case	Verify session artifact generation
Test Scenario	Verify that the Vision Service stores debug outputs and session logs for traceability
Input	Any valid screen frames submitted to relevant endpoint
Expected Output	Original image, processed image, JSON output, and logs should be stored

Actual Result	Session directory created successfully containing image copies, output JSON, and timing logs
---------------	--



and tabs with high confidence scores. The generated output included usable coordinates and descriptive metadata required for downstream automation modules.

Another important observation was the component's ability to generalize across previously unseen interfaces. Unlike rule-based or template-dependent systems, the Gemini-powered perception model successfully interpreted unfamiliar layouts without requiring predefined UI templates or hardcoded selectors.

The component also demonstrated operational stability by producing consistent structured JSON responses across repeated runs. Session artifacts such as processed frames, overlays, and logs were generated successfully, enabling traceability and debugging support.

Furthermore, the validation layer correctly handled uncertain conditions by filtering invalid coordinates, low-confidence detections, and malformed outputs. This confirms that the perception system operates not only as a detector but also as a reliable contextual intelligence service.

Figure 6.1 shows detected UI elements highlighted through overlay visualization.

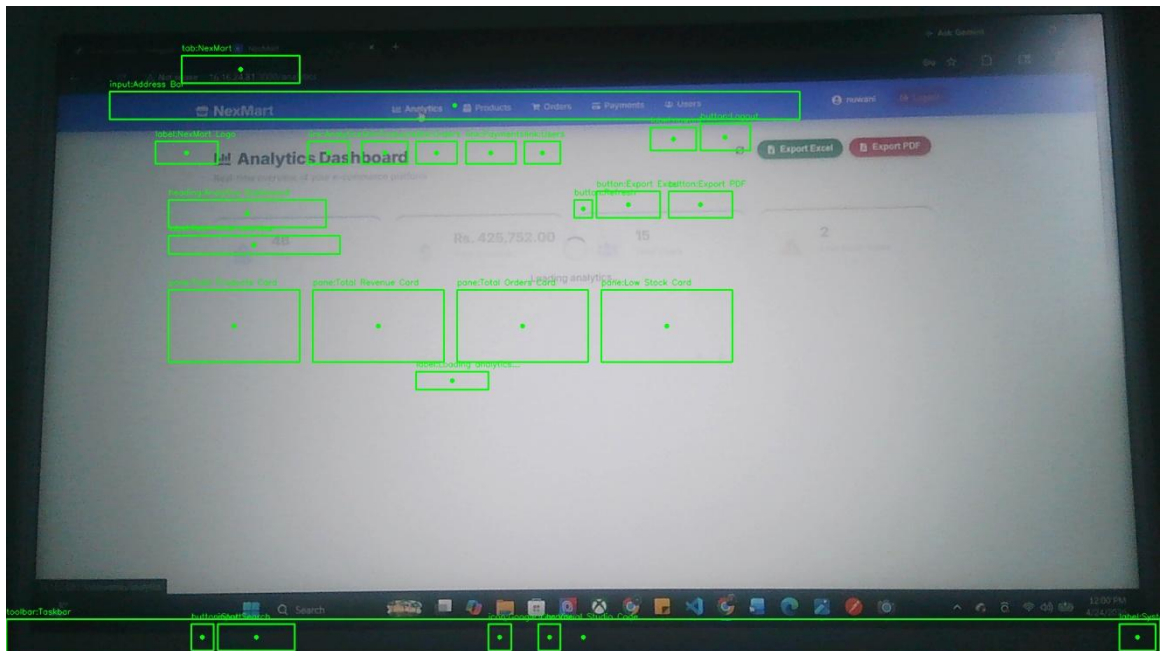


Figure 6.1: Detected UI Elements with Bounding overlays

Figure 6.2 shows a sample Postman request submitted to the Vision API. in terminal.

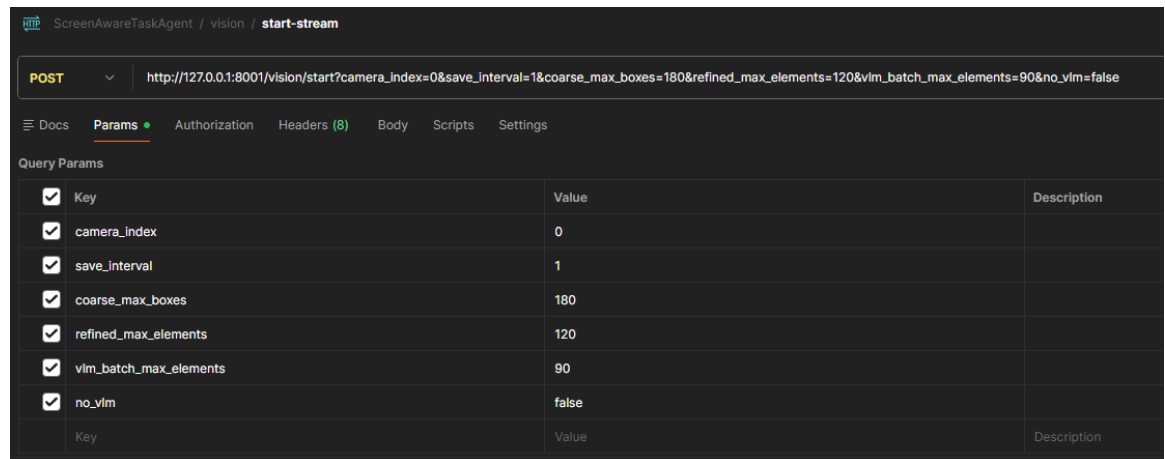


Figure 6.2: Vision API Request Payload through Postman

6.1.2 Quantitative Performance Summary

Experimental evaluations revealed measurable differences in performance depending on interface complexity, image quality, and perception model selection.

Simple interfaces with clear layouts and high-contrast controls were processed faster and with higher confidence. Examples include login forms, search bars, and browser toolbars. In these cases, the model quickly identified common UI semantics and produced outputs with minimal ambiguity.

Moderately complex interfaces such as dashboards, data-entry systems, and enterprise portals required more reasoning time because multiple visible elements competed for semantic relevance. Although latency increased slightly, detection quality remained strong.

Higher processing time was observed in cluttered or low-quality environments, such as blurred webcam captures, compressed screenshots, or highly dense dashboards. In such cases, the system still generated useful output but with slightly reduced confidence scores.

The observed performance values are summarized below.

Mode	Average Response Time
Gemini Production Mode	2.0 – 5.0 seconds
YOLO Prototype Mode	100 – 300 ms
Hybrid Prototype Mode	100 – 300 ms

Table 6.1: Vision Component Performance Summary

These results indicate a practical trade-off between speed and semantic quality. Faster local models offered low latency, while Gemini provided stronger generalized understanding suitable for production use.

6.1.3 Accuracy and Output Structure Quality

A key research outcome of the Visual Perception Component is the consistent generation of structured, machine-consumable outputs rather than simple descriptive text.

The produced JSON responses followed predefined schemas containing:

- Response status
- Detected element lists
- Element types
- Labels
- Coordinates
- Bounding boxes
- Confidence scores
- Processing timestamps

This consistency is essential because downstream planning systems require deterministic inputs for safe automation.

The quality of outputs can be evaluated through three dimensions:

Detection Accuracy

The system successfully identified common GUI components such as:

- Buttons

- Text fields
- Menus
- Tabs
- Icons
- Navigation bars
- Labels

In deterministic screens such as login forms, the model consistently identified all required elements.

Contextual Accuracy

Detected outputs aligned strongly with visible interface context. The system rarely generated unsupported or hallucinated coordinates. Elements were grounded in observable screen regions, confirming effective multimodal reasoning.

Execution Readiness

Coordinates and metadata were sufficiently accurate for downstream clicking, typing, and navigation tasks. This confirms that the perception output is operationally usable rather than merely descriptive.

Below shows a sample structured JSON output returned by the Vision Service. Note: In this sample output, the dx and dy coordinate values reflect the full screen resolution (1919 x 1079) rather than individual element positions. This is a known behaviour of the current Gemini output format where the model returns bounding context at the screen level. Element-level coordinate precision is an identified area for improvement in future iterations.

```
{
  "session_id": "session_1777012235_54c85a",
  "mode": "streaming",
  "started_at": "2026-04-24T12:00:35.239210",
  "finished_at": "2026-04-24T12:12:35.242137",
  "screen_count": 6,
  "screens": [
    {
      "screen_index": 0,
      "timestamp": "2026-04-24T12:01:02.730843",
```

```

    "image_path": "E:\\sliit-
project\\ScreenAwareTaskAgent\\vision\\data\\sessions\\session_1777012235_
54c85a\\processing\\proc_1777012237_8108da\\preprocessed_frames\\frame_177
7012237_ef0f83.jpg",
    "frame_hash":
"bbbe567ef06e155870cb687d114e50d40f732447956714e4674dda5822cb8017",
    "elements": [
        {
            "id": "browser_tab_nexmart",
            "type": "tab",
            "label": "NexMart",
            "description": "Browser tab for the NexMart application",
            "state": "active",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.95,
            "source": "gemini_vlm"
        },
        {
            "id": "address_bar",
            "type": "input",
            "label": "Address Bar",
            "description": "Browser address bar showing the URL
16.16.24.81:3000/analytics",
            "state": "visible",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.98,
            "source": "gemini_vlm"
        },
        {
            "id": "nexmart_logo",
            "type": "label",
            "label": "NexMart Logo",
            "description": "NexMart application logo and name",
            "state": "visible",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.95,
            "source": "gemini_vlm"
        },
        {
            "id": "nav_analytics",
            "type": "link",
            "label": "Analytics",
            "description": "Navigation link to the Analytics page",

```

```

    "state": "active",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_products",
    "type": "link",
    "label": "Products",
    "description": "Navigation link to the Products page",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_orders",
    "type": "link",
    "label": "Orders",
    "description": "Navigation link to the Orders page",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_payments",
    "type": "link",
    "label": "Payments",
    "description": "Navigation link to the Payments page",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_users",
    "type": "link",
    "label": "Users",
    "description": "Navigation link to the Users page",
    "state": "visible",
    "dx": 1919,

```



```

        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "user_profile",
        "type": "label",
        "label": "nuwani",
        "description": "Logged in user profile name",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "logout_button",
        "type": "button",
        "label": "Logout",
        "description": "Button to log out of the application",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "dashboard_heading",
        "type": "heading",
        "label": "Analytics Dashboard",
        "description": "Main heading of the analytics page",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.98,
        "source": "gemini_vlm"
    },
    {
        "id": "dashboard_subheading",
        "type": "label",
        "label": "Real-time overview",
        "description": "Subheading describing the dashboard content",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,

```

```

        "source": "gemini_vlm"
    },
    {
        "id": "refresh_button",
        "type": "button",
        "label": "Refresh",
        "description": "Icon button to refresh dashboard data",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "export_excel_button",
        "type": "button",
        "label": "Export Excel",
        "description": "Button to export data to Excel",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "export_pdf_button",
        "type": "button",
        "label": "Export PDF",
        "description": "Button to export data to PDF",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "summary_card_products",
        "type": "pane",
        "label": "Total Products Card",
        "description": "Summary card showing 46 total products",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    },

```

```

    {
      "id": "summary_card_revenue",
      "type": "pane",
      "label": "Total Revenue Card",
      "description": "Summary card showing Rs. 425,752.00 total
revenue",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "summary_card_orders",
      "type": "pane",
      "label": "Total Orders Card",
      "description": "Summary card showing 15 total orders",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "summary_card_low_stock",
      "type": "pane",
      "label": "Low Stock Card",
      "description": "Summary card showing 2 low stock items",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "loading_indicator",
      "type": "label",
      "label": "Loading analytics...",
      "description": "Text indicating that analytics data is being
loaded",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.95,
      "source": "gemini_vlm"
    },
  ],

```

```

{
  "id": "windows_taskbar",
  "type": "toolbar",
  "label": "Taskbar",
  "description": "Windows taskbar at the bottom of the screen",
  "state": "visible",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.98,
  "source": "gemini_vlm"
},
{
  "id": "start_button",
  "type": "button",
  "label": "Start",
  "description": "Windows Start button",
  "state": "visible",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.95,
  "source": "gemini_vlm"
},
{
  "id": "search_bar",
  "type": "input",
  "label": "Search",
  "description": "Windows taskbar search bar",
  "state": "visible",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.95,
  "source": "gemini_vlm"
},
{
  "id": "chrome_icon",
  "type": "icon",
  "label": "Google Chrome",
  "description": "Google Chrome icon on the taskbar",
  "state": "visible",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.95,
  "source": "gemini_vlm"
},
{
  "id": "vscode_icon",

```

```

        "type": "icon",
        "label": "Visual Studio Code",
        "description": "Visual Studio Code icon on the taskbar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "system_clock",
        "type": "label",
        "label": "System Clock",
        "description": "System clock and date in the taskbar tray",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    }
]
},
{
    "screen_index": 1,
    "timestamp": "2026-04-24T12:05:18.974444",
    "image_path": "E:\\sliit-
project\\ScreenAwareTaskAgent\\vision\\data\\sessions\\session_1777012235_
54c85a\\processing\\proc_1777012262_2b1284\\preprocessed_frames\\frame_177
7012238_4958c9.jpg",
    "frame_hash":
"ec2d5e60869d45b8ba62d146f060d33bc6df99d493473fff08009b86fa6f7092",
    "elements": [
        {
            "id": "text_loading_analytics",
            "type": "text",
            "label": "Loading analytics...",
            "description": "Text indicating that analytics are being
loaded.",
            "state": "visible",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.98,
            "source": "gemini_vlm"
        },
        {
            "id": "spinner_loading",

```

```

        "type": "icon",
        "label": "spinner",
        "description": "A circular loading spinner indicating a process
is in progress.",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "spinner_loading",
        "type": "icon",
        "label": "loading spinner",
        "description": "A circular loading spinner at the top left of
the screen.",
        "state": "active",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "0",
        "type": "text",
        "label": "NexMart",
        "description": "NexMart logo and home link",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "1",
        "type": "tab",
        "label": "Analytics",
        "description": "Analytics tab in the navigation bar",
        "state": "active",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "0",

```

```

        "type": "text",
        "label": "NexMart",
        "description": "NexMart logo text",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "1",
        "type": "tab",
        "label": "Analytics",
        "description": "Analytics navigation tab",
        "state": "selected",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "tab_analytics",
        "type": "tab",
        "label": "Analytics",
        "description": "Analytics tab in the navigation bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "tab_products",
        "type": "tab",
        "label": "Products",
        "description": "Products tab in the navigation bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"delta": {
    "added": [
        {

```

```

        "id": "text_loading_analytics",
        "type": "text",
        "label": "Loading analytics...",
        "description": "Text indicating that analytics are being
loaded.",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.98,
        "source": "gemini_vlm"
    },
    {
        "id": "tab_analytics",
        "type": "tab",
        "label": "Analytics",
        "description": "Analytics tab in the navigation bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "spinner_loading",
        "type": "icon",
        "label": "loading spinner",
        "description": "A circular loading spinner at the top left of
the screen.",
        "state": "active",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "0",
        "type": "text",
        "label": "NexMart",
        "description": "NexMart logo text",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {

```



```

        "id": "tab_products",
        "type": "tab",
        "label": "Products",
        "description": "Products tab in the navigation bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "1",
        "type": "tab",
        "label": "Analytics",
        "description": "Analytics navigation tab",
        "state": "selected",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"removed": [
    {
        "id": "logout_button",
        "type": "button",
        "label": "Logout",
        "description": "Button to log out of the application",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "nav_orders",
        "type": "link",
        "label": "Orders",
        "description": "Navigation link to the Orders page",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {

```

```

    "id": "chrome_icon",
    "type": "icon",
    "label": "Google Chrome",
    "description": "Google Chrome icon on the taskbar",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "summary_card_products",
    "type": "pane",
    "label": "Total Products Card",
    "description": "Summary card showing 46 total products",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "browser_tab_nexmart",
    "type": "tab",
    "label": "NexMart",
    "description": "Browser tab for the NexMart application",
    "state": "active",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_products",
    "type": "link",
    "label": "Products",
    "description": "Navigation link to the Products page",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "loading_indicator",
    "type": "label",

```

```

        "label": "Loading analytics...",
        "description": "Text indicating that analytics data is being
loaded",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "summary_card_revenue",
        "type": "pane",
        "label": "Total Revenue Card",
        "description": "Summary card showing Rs. 425,752.00 total
revenue",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "nav_payments",
        "type": "link",
        "label": "Payments",
        "description": "Navigation link to the Payments page",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "nav_analytics",
        "type": "link",
        "label": "Analytics",
        "description": "Navigation link to the Analytics page",
        "state": "active",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "user_profile",
        "type": "label",

```

```

        "label": "nuwani",
        "description": "Logged in user profile name",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "system_clock",
        "type": "label",
        "label": "System Clock",
        "description": "System clock and date in the taskbar tray",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "search_bar",
        "type": "input",
        "label": "Search",
        "description": "Windows taskbar search bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "export_pdf_button",
        "type": "button",
        "label": "Export PDF",
        "description": "Button to export data to PDF",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "dashboard_heading",
        "type": "heading",
        "label": "Analytics Dashboard",
        "description": "Main heading of the analytics page",

```

```

        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.98,
        "source": "gemini_vlm"
    },
    {
        "id": "start_button",
        "type": "button",
        "label": "Start",
        "description": "Windows Start button",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "vscode_icon",
        "type": "icon",
        "label": "Visual Studio Code",
        "description": "Visual Studio Code icon on the taskbar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "nexmart_logo",
        "type": "label",
        "label": "NexMart Logo",
        "description": "NexMart application logo and name",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "dashboard_subheading",
        "type": "label",
        "label": "Real-time overview",
        "description": "Subheading describing the dashboard content",
        "state": "visible",
        "dx": 1919,

```

```

        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "summary_card_orders",
        "type": "pane",
        "label": "Total Orders Card",
        "description": "Summary card showing 15 total orders",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "summary_card_low_stock",
        "type": "pane",
        "label": "Low Stock Card",
        "description": "Summary card showing 2 low stock items",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "windows_taskbar",
        "type": "toolbar",
        "label": "Taskbar",
        "description": "Windows taskbar at the bottom of the screen",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.98,
        "source": "gemini_vlm"
    },
    {
        "id": "address_bar",
        "type": "input",
        "label": "Address Bar",
        "description": "Browser address bar showing the URL
16.16.24.81:3000/analytics",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,

```

```

        "confidence": 0.98,
        "source": "gemini_vlm"
    },
    {
        "id": "export_excel_button",
        "type": "button",
        "label": "Export Excel",
        "description": "Button to export data to Excel",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "nav_users",
        "type": "link",
        "label": "Users",
        "description": "Navigation link to the Users page",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "refresh_button",
        "type": "button",
        "label": "Refresh",
        "description": "Icon button to refresh dashboard data",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"changed": []
}
},
{
    "screen_index": 2,
    "timestamp": "2026-04-24T12:05:52.690475",
    "image_path": "E:\\sliit-
project\\ScreenAwareTaskAgent\\vision\\data\\sessions\\session_1777012235_

```

```

54c85a\\processing\\proc_1777012293_d69f68\\preprocessed_frames\\frame_177
7012239_0cee76.jpg",
  "frame_hash":
"6bc455dddc77a38dfcc86453c53ddcdb5f90d120333d9b6e76f5401eabb13254",
  "elements": [
    {
      "id": "0",
      "type": "tab",
      "label": "NexMart Tab",
      "description": "Browser tab for NexMart application",
      "state": "active",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "1",
      "type": "input",
      "label": "Address Bar",
      "description": "Browser address bar showing the URL",
      "state": "enabled",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "browser_tab_nexmart",
      "type": "tab",
      "label": "NexMart Browser Tab",
      "description": "Browser tab for the NexMart application.",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "browser_ask_gemini",
      "type": "text",
      "label": "Ask Gemini",
      "description": "Browser feature text in the top right corner.",
      "state": "enabled",
      "dx": 1919,
      "dy": 1079,

```



```

        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "orders_by_status_heading",
        "type": "text",
        "label": "Orders by Status",
        "description": "Heading for the orders by status section",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "users_by_role_heading",
        "type": "text",
        "label": "Users by Role",
        "description": "Heading for the users by role section",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"delta": {
    "added": [
        {
            "id": "users_by_role_heading",
            "type": "text",
            "label": "Users by Role",
            "description": "Heading for the users by role section",
            "state": "visible",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.9,
            "source": "gemini_vlm"
        },
        {
            "id": "browser_ask_gemini",
            "type": "text",
            "label": "Ask Gemini",
            "description": "Browser feature text in the top right
corner.",
            "state": "enabled",

```

```

        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "browser_tab_nexmart",
        "type": "tab",
        "label": "NexMart Browser Tab",
        "description": "Browser tab for the NexMart application.",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "orders_by_status_heading",
        "type": "text",
        "label": "Orders by Status",
        "description": "Heading for the orders by status section",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"removed": [
    {
        "id": "text_loading_analytics",
        "type": "text",
        "label": "Loading analytics...",
        "description": "Text indicating that analytics are being
loaded.",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.98,
        "source": "gemini_vlm"
    },
    {
        "id": "spinner_loading",
        "type": "icon",
        "label": "loading spinner",

```

```

        "description": "A circular loading spinner at the top left of
the screen.",
        "state": "active",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "tab_products",
        "type": "tab",
        "label": "Products",
        "description": "Products tab in the navigation bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "tab_analytics",
        "type": "tab",
        "label": "Analytics",
        "description": "Analytics tab in the navigation bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"changed": [
    {
        "id": "1",
        "before": {
            "type": "tab",
            "label": "Analytics",
            "state": "selected"
        },
        "after": {
            "type": "input",
            "label": "Address Bar",
            "state": "enabled"
        }
    }
],
{

```

```

        "id": "0",
        "before": {
            "type": "text",
            "label": "NexMart",
            "state": "visible"
        },
        "after": {
            "type": "tab",
            "label": "NexMart Tab",
            "state": "active"
        }
    }
}
],
},
{
    "screen_index": 3,
    "timestamp": "2026-04-24T12:09:23.984349",
    "image_path": "E:\\sliit-
project\\ScreenAwareTaskAgent\\vision\\data\\sessions\\session_1777012235_
54c85a\\processing\\proc_1777012518_657431\\preprocessed_frames\\frame_177
7012242_4acf4a.jpg",
    "frame_hash":
"8894b62f87a57dddaa98923307823e3ac592aa21c0bd69fd8b60c112b69305b5",
    "elements": [
        {
            "id": "nexmart_logo",
            "type": "label",
            "label": "NexMart",
            "description": "NexMart application logo and brand name",
            "state": "normal",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.95,
            "source": "gemini_vlm"
        },
        {
            "id": "nav_products",
            "type": "link",
            "label": "Products",
            "description": "Navigation link to Products page",
            "state": "active",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.95,
            "source": "gemini_vlm"
        }
    ]
}

```

```

    },
    {
      "id": "heading_products_management",
      "type": "heading",
      "label": "Products Management",
      "description": "Main heading of the products management page",
      "state": "normal",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.95,
      "source": "gemini_vlm"
    },
    {
      "id": "taskbar",
      "type": "taskbar",
      "label": "taskbar",
      "description": "Windows taskbar",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.95,
      "source": "gemini_vlm"
    },
    {
      "id": "browser_address_bar",
      "type": "input",
      "label": "Address Bar",
      "description": "Browser address bar showing the URL
16.16.24.81:3000/admin/products",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "browser_tab_1",
      "type": "tab",
      "label": "NexMart",
      "description": "Browser tab for NexMart application",
      "state": "inactive",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
  ],

```

```

{
  "id": "browser_tab_2",
  "type": "tab",
  "label": "NexMart",
  "description": "Active browser tab for NexMart application",
  "state": "active",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.9,
  "source": "gemini_vlm"
},
{
  "id": "address_bar",
  "type": "input",
  "label": "Address Bar",
  "description": "Browser address bar showing the URL",
  "state": "normal",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.9,
  "source": "gemini_vlm"
},
{
  "id": "nav_analytics",
  "type": "link",
  "label": "Analytics",
  "description": "Navigation link to Analytics page",
  "state": "normal",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.9,
  "source": "gemini_vlm"
},
{
  "id": "nav_orders",
  "type": "link",
  "label": "Orders",
  "description": "Navigation link to Orders page",
  "state": "normal",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.9,
  "source": "gemini_vlm"
},
{
  "id": "nav_payments",

```

```

    "type": "link",
    "label": "Payments",
    "description": "Navigation link to Payments page",
    "state": "normal",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_users",
    "type": "link",
    "label": "Users",
    "description": "Navigation link to Users page",
    "state": "normal",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "btn_export",
    "type": "button",
    "label": "Export",
    "description": "Button to export product data",
    "state": "normal",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "loading_text",
    "type": "label",
    "label": "Loading...",
    "description": "Text indicating that content is being loaded",
    "state": "normal",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "text_ask_gemini",
    "type": "text",
    "label": "Ask Gemini",

```

```

    "description": "Text Ask Gemini in the top bar",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "loading_text",
    "type": "label",
    "label": "Loading...",
    "description": "Loading text at the top of the screen",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "url_tooltip",
    "type": "label",
    "label": "http://localhost:3000/admin/products",
    "description": "URL tooltip at the bottom left",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "loading_text",
    "type": "text",
    "label": "text",
    "description": "Loading...",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "icon_settings",
    "type": "icon",
    "label": "Settings",
    "description": "Settings icon in the top bar",
    "state": "visible",

```



```

        "dx": 1919,
        "dy": 1079,
        "confidence": 0.8,
        "source": "gemini_vlm"
    }
],
"delta": {
    "added": [
        {
            "id": "loading_text",
            "type": "text",
            "label": "text",
            "description": "Loading...",
            "state": "visible",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.9,
            "source": "gemini_vlm"
        },
        {
            "id": "nav_orders",
            "type": "link",
            "label": "Orders",
            "description": "Navigation link to Orders page",
            "state": "normal",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.9,
            "source": "gemini_vlm"
        },
        {
            "id": "browser_address_bar",
            "type": "input",
            "label": "Address Bar",
            "description": "Browser address bar showing the URL
16.16.24.81:3000/admin/products",
            "state": "visible",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.9,
            "source": "gemini_vlm"
        },
        {
            "id": "icon_settings",
            "type": "icon",
            "label": "Settings",

```

```

        "description": "Settings icon in the top bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.8,
        "source": "gemini_vlm"
    },
    {
        "id": "browser_tab_2",
        "type": "tab",
        "label": "NexMart",
        "description": "Active browser tab for NexMart application",
        "state": "active",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "text_ask_gemini",
        "type": "text",
        "label": "Ask Gemini",
        "description": "Text Ask Gemini in the top bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "nav_payments",
        "type": "link",
        "label": "Payments",
        "description": "Navigation link to Payments page",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "heading_products_management",
        "type": "heading",
        "label": "Products Management",
        "description": "Main heading of the products management page",
        "state": "normal",

```

```

        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "url_tooltip",
        "type": "label",
        "label": "http://localhost:3000/admin/products",
        "description": "URL tooltip at the bottom left",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "btn_export",
        "type": "button",
        "label": "Export",
        "description": "Button to export product data",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "address_bar",
        "type": "input",
        "label": "Address Bar",
        "description": "Browser address bar showing the URL",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "nav_products",
        "type": "link",
        "label": "Products",
        "description": "Navigation link to Products page",
        "state": "active",
        "dx": 1919,
        "dy": 1079,

```

```

        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "taskbar",
        "type": "taskbar",
        "label": "taskbar",
        "description": "Windows taskbar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "nav_users",
        "type": "link",
        "label": "Users",
        "description": "Navigation link to Users page",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "nexmart_logo",
        "type": "label",
        "label": "NexMart",
        "description": "NexMart application logo and brand name",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.95,
        "source": "gemini_vlm"
    },
    {
        "id": "browser_tab_1",
        "type": "tab",
        "label": "NexMart",
        "description": "Browser tab for NexMart application",
        "state": "inactive",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }

```

```

    },
    {
      "id": "nav_analytics",
      "type": "link",
      "label": "Analytics",
      "description": "Navigation link to Analytics page",
      "state": "normal",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    }
  ],
  "removed": [
    {
      "id": "0",
      "type": "tab",
      "label": "NexMart Tab",
      "description": "Browser tab for NexMart application",
      "state": "active",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "users_by_role_heading",
      "type": "text",
      "label": "Users by Role",
      "description": "Heading for the users by role section",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "browser_ask_gemini",
      "type": "text",
      "label": "Ask Gemini",
      "description": "Browser feature text in the top right
corner.",
      "state": "enabled",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,

```

```

        "source": "gemini_vlm"
    },
    {
        "id": "1",
        "type": "input",
        "label": "Address Bar",
        "description": "Browser address bar showing the URL",
        "state": "enabled",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "browser_tab_nexmart",
        "type": "tab",
        "label": "NexMart Browser Tab",
        "description": "Browser tab for the NexMart application.",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "orders_by_status_heading",
        "type": "text",
        "label": "Orders by Status",
        "description": "Heading for the orders by status section",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"changed": []
}
},
{
    "screen_index": 4,
    "timestamp": "2026-04-24T12:09:54.140580",
    "image_path": "E:\\sliit-
project\\ScreenAwareTaskAgent\\vision\\data\\sessions\\session_1777012235_
54c85a\\processing\\proc_1777012552_b64973\\preprocessed_frames\\frame_177
7012249_87c267.jpg",

```

```

    "frame_hash":
    "fdb5d5cc143f93f7ec542831ba54fc14a9cdff177650ec79c5cbd1462bd5064b",
    "elements": [
      {
        "id": "0",
        "type": "tab",
        "label": "NexMart",
        "description": "Browser tab for NexMart application",
        "state": "active",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
      },
      {
        "id": "1",
        "type": "input",
        "label": "Address Bar",
        "description": "Browser address bar showing the URL",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
      },
      {
        "id": "nexmart_logo",
        "type": "text",
        "label": "NexMart",
        "description": "NexMart logo and brand name in the navigation
bar.",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
      },
      {
        "id": "ask_gemini_text",
        "type": "text",
        "label": "Ask Gemini",
        "description": "Text label for Ask Gemini feature",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,

```

```

        "source": "gemini_vlm"
    },
    {
        "id": "star_icon",
        "type": "icon",
        "label": "Star",
        "description": "Star icon in the top bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "windows_icon",
        "type": "icon",
        "label": "Windows Start",
        "description": "Windows start menu icon on the taskbar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "category_header",
        "type": "text",
        "label": "CATEGORY",
        "description": "Table header for category column",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "price_header",
        "type": "text",
        "label": "PRICE",
        "description": "Table header for price column",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
}

```



```

],
"delta": {
  "added": [
    {
      "id": "category_header",
      "type": "text",
      "label": "CATEGORY",
      "description": "Table header for category column",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "price_header",
      "type": "text",
      "label": "PRICE",
      "description": "Table header for price column",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "0",
      "type": "tab",
      "label": "NexMart",
      "description": "Browser tab for NexMart application",
      "state": "active",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "1",
      "type": "input",
      "label": "Address Bar",
      "description": "Browser address bar showing the URL",
      "state": "normal",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    }
  ]
}

```

```

    },
    {
      "id": "windows_icon",
      "type": "icon",
      "label": "Windows Start",
      "description": "Windows start menu icon on the taskbar",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "star_icon",
      "type": "icon",
      "label": "Star",
      "description": "Star icon in the top bar",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "ask_gemini_text",
      "type": "text",
      "label": "Ask Gemini",
      "description": "Text label for Ask Gemini feature",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    }
  ],
  "removed": [
    {
      "id": "loading_text",
      "type": "text",
      "label": "text",
      "description": "Loading...",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    }
  ]
}

```

```

    },
    {
      "id": "nav_orders",
      "type": "link",
      "label": "Orders",
      "description": "Navigation link to Orders page",
      "state": "normal",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "browser_address_bar",
      "type": "input",
      "label": "Address Bar",
      "description": "Browser address bar showing the URL
16.16.24.81:3000/admin/products",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
    {
      "id": "icon_settings",
      "type": "icon",
      "label": "Settings",
      "description": "Settings icon in the top bar",
      "state": "visible",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.8,
      "source": "gemini_vlm"
    },
    {
      "id": "browser_tab_2",
      "type": "tab",
      "label": "NexMart",
      "description": "Active browser tab for NexMart application",
      "state": "active",
      "dx": 1919,
      "dy": 1079,
      "confidence": 0.9,
      "source": "gemini_vlm"
    },
  ],

```

```

{
  "id": "text_ask_gemini",
  "type": "text",
  "label": "Ask Gemini",
  "description": "Text Ask Gemini in the top bar",
  "state": "visible",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.9,
  "source": "gemini_vlm"
},
{
  "id": "heading_products_management",
  "type": "heading",
  "label": "Products Management",
  "description": "Main heading of the products management page",
  "state": "normal",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.95,
  "source": "gemini_vlm"
},
{
  "id": "url_tooltip",
  "type": "label",
  "label": "http://localhost:3000/admin/products",
  "description": "URL tooltip at the bottom left",
  "state": "visible",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.9,
  "source": "gemini_vlm"
},
{
  "id": "btn_export",
  "type": "button",
  "label": "Export",
  "description": "Button to export product data",
  "state": "normal",
  "dx": 1919,
  "dy": 1079,
  "confidence": 0.9,
  "source": "gemini_vlm"
},
{
  "id": "address_bar",

```

```

    "type": "input",
    "label": "Address Bar",
    "description": "Browser address bar showing the URL",
    "state": "normal",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_products",
    "type": "link",
    "label": "Products",
    "description": "Navigation link to Products page",
    "state": "active",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "taskbar",
    "type": "taskbar",
    "label": "taskbar",
    "description": "Windows taskbar",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.95,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_users",
    "type": "link",
    "label": "Users",
    "description": "Navigation link to Users page",
    "state": "normal",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "nav_payments",
    "type": "link",
    "label": "Payments",

```

```

        "description": "Navigation link to Payments page",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "browser_tab_1",
        "type": "tab",
        "label": "NexMart",
        "description": "Browser tab for NexMart application",
        "state": "inactive",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "nav_analytics",
        "type": "link",
        "label": "Analytics",
        "description": "Navigation link to Analytics page",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"changed": [
    {
        "id": "nexmart_logo",
        "before": {
            "type": "label",
            "label": "NexMart",
            "state": "normal"
        },
        "after": {
            "type": "text",
            "label": "NexMart",
            "state": "visible"
        }
    }
]
}

```

```

    },
    {
      "screen_index": 5,
      "timestamp": "2026-04-24T12:12:35.229425",
      "image_path": "E:\\sliit-
project\\ScreenAwareTaskAgent\\vision\\data\\sessions\\session_1777012235_
54c85a\\processing\\proc_1777012763_bd5e50\\preprocessed_frames\\frame_177
7012251_9420fd.jpg",
      "frame_hash":
"6a2bec535d57ad2910a77b7c07bcf34a3d834e6fac41af80e5a21c6cccdc2384",
      "elements": [
        {
          "id": "nexmart_logo",
          "type": "link",
          "label": "NexMart",
          "description": "NexMart logo and home link",
          "state": "visible",
          "dx": 1919,
          "dy": 1079,
          "confidence": 0.9,
          "source": "gemini_vlm"
        },
        {
          "id": "analytics_tab",
          "type": "tab",
          "label": "Analytics",
          "description": "Analytics navigation tab",
          "state": "visible",
          "dx": 1919,
          "dy": 1079,
          "confidence": 0.9,
          "source": "gemini_vlm"
        },
        {
          "id": "analytics_tab",
          "type": "tab",
          "label": "Analytics",
          "description": "Analytics navigation tab",
          "state": "inactive",
          "dx": 1919,
          "dy": 1079,
          "confidence": 0.9,
          "source": "gemini_vlm"
        },
        {
          "id": "products_tab",

```

```

    "type": "tab",
    "label": "Products",
    "description": "Products navigation tab",
    "state": "inactive",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "order_1_heading",
    "type": "text",
    "label": "Order #69ea219b33b46455f2d2159c",
    "description": "Heading for the first order entry",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "0",
    "type": "text",
    "label": "PRODUCT:",
    "description": "Label for the product ID",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "1",
    "type": "text",
    "label": "69ea2165cfe5efad628dc6bb",
    "description": "Product ID string",
    "state": "visible",
    "dx": 1919,
    "dy": 1079,
    "confidence": 0.9,
    "source": "gemini_vlm"
  },
  {
    "id": "browser_tabs",
    "type": "pane",
    "label": "Browser Tabs",

```



```

        "description": "Browser tab bar showing open pages",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.8,
        "source": "gemini_vlm"
    },
    {
        "id": "address_bar",
        "type": "input",
        "label": "Address Bar",
        "description": "Browser address and search bar",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.8,
        "source": "gemini_vlm"
    }
],
"delta": {
    "added": [
        {
            "id": "analytics_tab",
            "type": "tab",
            "label": "Analytics",
            "description": "Analytics navigation tab",
            "state": "inactive",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.9,
            "source": "gemini_vlm"
        },
        {
            "id": "order_1_heading",
            "type": "text",
            "label": "Order #69ea219b33b46455f2d2159c",
            "description": "Heading for the first order entry",
            "state": "visible",
            "dx": 1919,
            "dy": 1079,
            "confidence": 0.9,
            "source": "gemini_vlm"
        },
        {
            "id": "browser_tabs",
            "type": "pane",

```

```

        "label": "Browser Tabs",
        "description": "Browser tab bar showing open pages",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.8,
        "source": "gemini_vlm"
    },
    {
        "id": "address_bar",
        "type": "input",
        "label": "Address Bar",
        "description": "Browser address and search bar",
        "state": "normal",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.8,
        "source": "gemini_vlm"
    },
    {
        "id": "products_tab",
        "type": "tab",
        "label": "Products",
        "description": "Products navigation tab",
        "state": "inactive",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"removed": [
    {
        "id": "category_header",
        "type": "text",
        "label": "CATEGORY",
        "description": "Table header for category column",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "price_header",
        "type": "text",

```

```

        "label": "PRICE",
        "description": "Table header for price column",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "windows_icon",
        "type": "icon",
        "label": "Windows Start",
        "description": "Windows start menu icon on the taskbar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "star_icon",
        "type": "icon",
        "label": "Star",
        "description": "Star icon in the top bar",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    },
    {
        "id": "ask_gemini_text",
        "type": "text",
        "label": "Ask Gemini",
        "description": "Text label for Ask Gemini feature",
        "state": "visible",
        "dx": 1919,
        "dy": 1079,
        "confidence": 0.9,
        "source": "gemini_vlm"
    }
],
"changed": [
    {
        "id": "nexmart_logo",
        "before": {

```

```

        "type": "text",
        "label": "NexMart",
        "state": "visible"
    },
    "after": {
        "type": "link",
        "label": "NexMart",
        "state": "visible"
    }
},
{
    "id": "1",
    "before": {
        "type": "input",
        "label": "Address Bar",
        "state": "normal"
    },
    "after": {
        "type": "text",
        "label": "69ea2165cfe5efad628dc6bb",
        "state": "visible"
    }
},
{
    "id": "0",
    "before": {
        "type": "tab",
        "label": "NexMart",
        "state": "active"
    },
    "after": {
        "type": "text",
        "label": "PRODUCT:",
        "state": "visible"
    }
}
]
}
]
}
]
}

```

6.1.4 Robustness Under Edge Condition

Robustness evaluation examined system behavior under uncertain and non-ideal operating conditions.

Missing Element Conditions

When expected controls were absent, the system generally avoided false-positive detections and instead returned only available visible elements. For example, if a login button was hidden or outside the camera frame, the model did not fabricate a button unnecessarily.

Low-Quality Images

The system retained acceptable usability under moderate blur, glare, and low-light webcam captures. Confidence scores reduced slightly, but primary interface objects remained detectable.

Dense Interface Conditions

In dashboards with many visible widgets and controls, the system prioritized semantically relevant elements such as search bars, side menus, and action buttons.

Output Transparency

Even under uncertain conditions, the component returned explicit confidence values and structured outputs, allowing downstream modules to make threshold-based decisions.

This robustness is valuable for real-world deployment where screen conditions are rarely ideal.

6.1.5 Comparative Behavior Across Scenario Types

Comparative testing across multiple interface categories revealed clear performance patterns.

Structured Login / Form Interfaces

These scenarios achieved the highest reliability and lowest ambiguity. Input fields and buttons were consistently detected.

Browser Interfaces

Tabs, address bars, search controls, and navigation buttons were detected successfully, though crowded tab regions slightly increased reasoning complexity.

Dashboard Interfaces

Detection remained strong, but processing time increased due to higher element density.

Webcam Black-Box Capture Scenarios

External camera capture introduced perspective distortion and glare, yet the system still produced usable outputs after preprocessing.

Overall, the component performed best in visually structured and predictable environments while remaining functional in noisier real-world conditions.

6.1.6 Practical Result Interpretation

From a practical deployment perspective, the Visual Perception Component is suitable for automation tasks where:

- Visible controls are clearly rendered
- Interface state changes are moderate
- User actions depend on screen understanding
- No internal software integration exists

This makes the system highly applicable for:

- Legacy desktop systems

- Third-party enterprise tools
- Restricted environments
- Black-box automation scenarios
- Cross-platform UI assistance systems

The results confirm that multimodal perception can replace many selector-based approaches where source-code access is unavailable.

6.2 Research Finding

The following key findings were derived from implementation and experimental evaluation.

6.2.1 Visual-language reasoning enables practical screen understanding

Gemini-based multimodal analysis successfully interpreted real software interfaces and converted them into structured automation context.

6.2.2 Structured output improves downstream integration

Coordinates, labels, confidence values, and JSON schemas made perception results directly usable by planning systems.

6.2.3 Validation layers increase reliability

Coordinate checks, schema filtering, and confidence thresholds reduced unsafe outputs.

6.2.4 Semantic generalization outperformed fixed detectors

Compared with YOLO-only approaches, Gemini handled unseen interfaces more effectively.

6.2.5 Speed-quality trade-offs exist

Fast local detectors were quicker, but Gemini delivered higher semantic accuracy.

6.2.6 Black-box automation is feasible

The webcam-based perception approach demonstrated that automation could occur without installing software on the target computer.

6.2.7 Modular architecture supports extensibility

The service can later integrate OCR upgrades, faster models, caching, or hybrid routing without redesign.

6.3 Discussion

6.3.1 Interpretation Against Research Objectives

The primary objective of this research was to design and evaluate a visual perception framework capable of understanding software interfaces in black-box environments. Experimental outcomes confirm that this objective was successfully achieved.

The component consistently transformed raw images into structured actionable context suitable for intelligent automation.

6.3.2 Effectiveness of the Proposed Approach

The effectiveness of architecture arises from combining deterministic engineering with probabilistic AI reasoning.

- OpenCV ensures stable preprocessing
- Gemini provides semantic understanding
- Validation layers enforce output reliability
- JSON formatting enables integration

This hybrid methodology balances flexibility with engineering control.

6.3.3 Implication of Human-Computer Interaction

This work demonstrates a new interaction model where systems can visually observe interfaces rather than requiring deep integration of APIs.

This has implications for:

- Accessibility tools
- Assistive computing
- Legacy software automation

- Cross-platform intelligent agents
- Low-code automation systems

6.3.4 Reliability-Latency Trade-off

A clear trade-off was observed between semantic quality and inference speed.

Gemini required more processing time than YOLO, but delivered:

- Better UI understanding
- Improved generalization
- More usable structured outputs

Future systems may combine fast local routing with Gemini fallback.

6.3.5 Workflow Continuity as a Limitation and Opportunity

The component still faces limitations such as:

- Latency under cloud inference
- Reduced confidence under severe blur
- Difficulty in highly dynamic animated interfaces
- Need for better state continuity across multiple screens

Future enhancements may include:

- Hybrid YOLO + Gemini routing
- Local multimodal acceleration
- Memory-based state tracking
- Adaptive confidence policies

6.3.6 Trust, Safety and Explainability

The system improves transparency through:

- Confidence scores
- Structured JSON outputs

- Debug overlays
- Session logs
- Explicit coordinate data

These features are valuable for auditing and safe deployment.

6.3.7 Threats to Validity

Findings are subject to several limitations:

- Limited evaluation scenarios
- Dependence on image quality
- Dependence on Gemini behavior
- Variability across future UI styles

However, results strongly support feasibility of the proposed approach.

6.3.8 Practical Deployment Implications

The component is suitable for deployment in:

- Enterprise form automation
- Legacy application assistance
- Browser workflow automation
- Remote screen assistance
- Human-supervised robotic UI control systems

7. CONCLUSIONS

This research set out to design and validate a Visual Perception Component capable of understanding graphical user interfaces through image-based screen analysis using multimodal AI reasoning. The overall objective was to determine whether a vision-driven perception pipeline could provide accurate, structured, and reliable screen understanding

for practical automation tasks in real-world computing environments. Based on the implementation and evaluation results, this objective was successfully achieved.

The developed system demonstrated a complete end-to-end perception workflow. Raw screenshots or live captured frames were received through the Vision Service, preprocessed using image enhancement techniques, analyzed through the Gemini multimodal model, validated through schema and coordinate checking layers, and finally transformed into structured JSON outputs containing detected interface elements, labels, confidence values, and actionable coordinates. This confirms that the proposed architecture is not only theoretically sound but also operationally practical for modern screen-aware automation systems.

The experimental outcomes showed that the component can accurately detect and classify common user interface objects such as buttons, text fields, menus, labels, tabs, icons, browser controls, navigation panes, and other visible interface components. These detections were consistently returned in structured formats suitable for downstream automation modules such as planners, command generators, and interaction agents. As a result, the component successfully bridges the gap between raw visual input and machine-executable contextual intelligence.

A key contribution of this work is the adoption of a generalized multimodal reasoning approach rather than relying solely on fixed object detectors or hardcoded templates. Earlier experiments with local VLM models and YOLO-based detection pipelines demonstrated speed advantages but limited generalization across unseen interfaces. In contrast, production Gemini-based implementation provided stronger semantic understanding, improved adaptability, and more reliable detection performance across diverse software environments. This design decision significantly increased the scalability and practical applicability of the component.

Another important conclusion is that high-quality structured outputs can be generated directly from visual input without requiring internal application APIs, DOM access, or software instrumentation. This confirms the feasibility of black-box automation, where systems can understand and interact with external software environments using only

screen visibility. Such capability is particularly valuable for legacy systems, third-party applications, restricted enterprise platforms, and cross-device automation scenarios.

From a research perspective, the study confirms the central hypothesis that vision-language reasoning can serve as a dependable perception layer for intelligent task automation. The modular decomposition of the system into API handling, preprocessing, multimodal inference, validation, artifact logging, and structured response generation also provides strong extensibility for future enhancements. Individual layers can be improved independently without redesigning the full architecture, thereby supporting maintainability and long-term evolution.

Despite these strengths, the work also identified several improvement opportunities. Cloud-based multimodal inference introduced measurable latency compared with earlier local detection approaches. Performance also varied under blurred images, cluttered dashboards, and highly dynamic interfaces. In addition, long-horizon state tracking across multiple screen transitions remains an open challenge for future perception systems. These limitations do not weaken the core contribution; instead, they define a clear roadmap for continued development.

Future enhancements may include hybrid YOLO and Gemini routing strategies, local multimodal acceleration, adaptive confidence policies, improved OCR fusion, temporal memory across screens, and real-time optimization techniques. Such improvements would further strengthen responsiveness, robustness, and deployment readiness in enterprise-scale environments.

In summary, this research contributes a practical and technically validated foundation for intelligent screen understanding through AI-powered visual perception. The implemented Visual Perception Component demonstrates that combining computer vision engineering with multimodal reasoning can produce accurate, interpretable, and automation-ready contextual outputs. Future work can build directly upon this foundation to improve speed, continuity, and large-scale deployment across increasingly complex digital environments.

8. REFERENCES

- [1] T. Yeh, T. Chang and R. C. Miller, "Sikuli: Using GUI Screenshots for Search and Automation," in *Proceedings of the 22nd Annual ACM Symposium on User Interface Software and Technology (UIST)*, pp. 183–192, 2009.
- [2] K. Larson, "Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence," *ScreenAgent: A Vision Language Model-driven Computer Control Agent*, 2024.
- [3] Z. Z. and A. Z. , "You Only Look at Screens: Multimodal Chain-of-Action Agents," Jun. 2024.
- [4] X. M. Z. Z. and H. Z. , "CoCo-Agent: A Comprehensive Cognitive MLLM Agent for Smartphone GUI Automation," Jun. 2024.
- [5] R. Smith, "in Ninth International Conference on Document Analysis and Recognition (ICDAR 2007), IEEE," *An Overview of the Tesseract OCR Engine*, p. pp. 629–633, Sep. 2007.
- [6] J. C. e. al, "Object Detection for Graphical User Interface: Old Fashioned or Deep Learning or a Combination?," Sep. 2020.
- [7] H. Liu, C. Li, Q. Wu and Y. J. Lee, "Visual Instruction Tuning," p. 25, Dec. 2023.
- [8] R. L. L. W. W. H. and X. X. , "GUI-R1: A Generalist R1-Style Vision-Language Action Model for GUI Agents," Apr. 2025.
- [9] P. P. e. al, "TinyClick: Single-Turn Agent for Empowering GUI Automation," May. 2025.
- [10] W. Hong et al., "CogAgent: A Visual Language Model for GUI Agents," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [11] I. Altaf, J. A. Dar, F. u. Rashid and M. Rafiq, "Survey on Selenium Tool in Software Testing," 2015.

- [12] A. A. Alotaibi and R. J. Qureshi, "Novel Framework for Automation Testing of Mobile Applications using Appium," p. pp. 34–40, 2017.
- [13] X. M. Wang, X. Huang, G. Zhi L, C. P. Miao and N. Shen, "Research of Mobile Applications Automated Testing Using UIAutomator," 2016.
- [14] "SikuliX, "SikuliX Automation Tool," [Online]. Available: <https://sikulix.com/>.
- [15] JaiedAI, "EasyOCR: Ready-to-use OCR with 80+ Supported Languages," [Online]. Available: <https://github.com/JaiedAI/EasyOCR>.
- [16] R. Smith et al., "Tesseract OCR GitHub Repository," [Online]. Available: <https://github.com/tesseract-ocr/tesseract>.
- [17] "PaddlePaddle, "PaddleOCR GitHub Repository," [Online]. Available: <https://github.com/PaddlePaddle/PaddleOCR>.
- [18] "SeleniumHQ, "Selenium Official Documentation," [Online]. Available: <https://www.selenium.dev/documentation/>.
- [19] "Appium, "Appium Official Documentation," [Online]. Available: <https://appium.io/docs/en/latest/>.
- [20] "Google Developers, "UIAutomator Documentation.,"" [Online]. Available: <https://developer.android.com/training/testing/ui-automator>.
- [21] H. L. e. al, "LLaVA: Visual Instruction Tuning GitHub Repository," [Online]. Available: <https://github.com/haotian-liu/LLaVA>.
- [22] "THUDM, "CogAgent GitHub Repository.,"" [Online]. Available: <https://github.com/THUDM/CogAgent>.
- [23] "Ultralytics, "YOLOv8 Documentation.,"" [Online]. Available: <https://docs.ultralytics.com/>.
- [24] "Google Developers, "Gemini API Documentation.,"" [Online]. Available: <https://ai.google.dev/>.
- [25] "OpenAI, "GPT-4 Technical Report," 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>.

- [26] "Microsoft, "Windows UI Automation Overview."," [Online]. Available: <https://learn.microsoft.com/>.
- [27] "OpenCV, "OpenCV Official Documentation."," [Online]. Available: <https://opencv.org/>.
- [28] "NumPy, "NumPy Official Documentation."," [Online]. Available: <https://numpy.org/>.
- [29] "FastAPI, "FastAPI Official Documentation."," [Online]. Available: <https://fastapi.tiangolo.com/>.
- [30] "Python Software Foundation, "Python Official Documentation."," [Online]. Available: <https://docs.python.org/3/>.

9. APPENDICES

Appendix - A: Plagiarism Report

The screenshot shows the Turnitin 'Submit work' interface for a course titled 'Undergrad Thesis Checking'. The page includes a header with the Turnitin logo and user navigation links. Below the header, there's a section for 'Submit work' with details about the assignment, including start and due dates, max points, and submission rules. A table titled 'Your work' shows a submission for the file 'IT22099518-final-report-draft' with a similarity score of 4%.

Submit work			
Undergrad Thesis Checking			
Title	Start date	Due date	Feedback release date
Undergrad Thesis Checking	Apr 06, 2026, 12:00 AM	Dec 31, 2027, 11:59 PM	Apr 20, 2026, 11:59 PM
Instructions	Max points	Rubric	
Template	100	(Not set)	
No template uploaded			
❌ Late submissions are not allowed ✅ Resubmissions are allowed			
Your work			
Title	Submitted	Grade	Similarity
IT22099518-final-report-draft	Apr 26, 2026 8:44 AM	-- / 100	4%